



Day 1: GPU Computing in Python

USING PYCUDA AND NUMBA

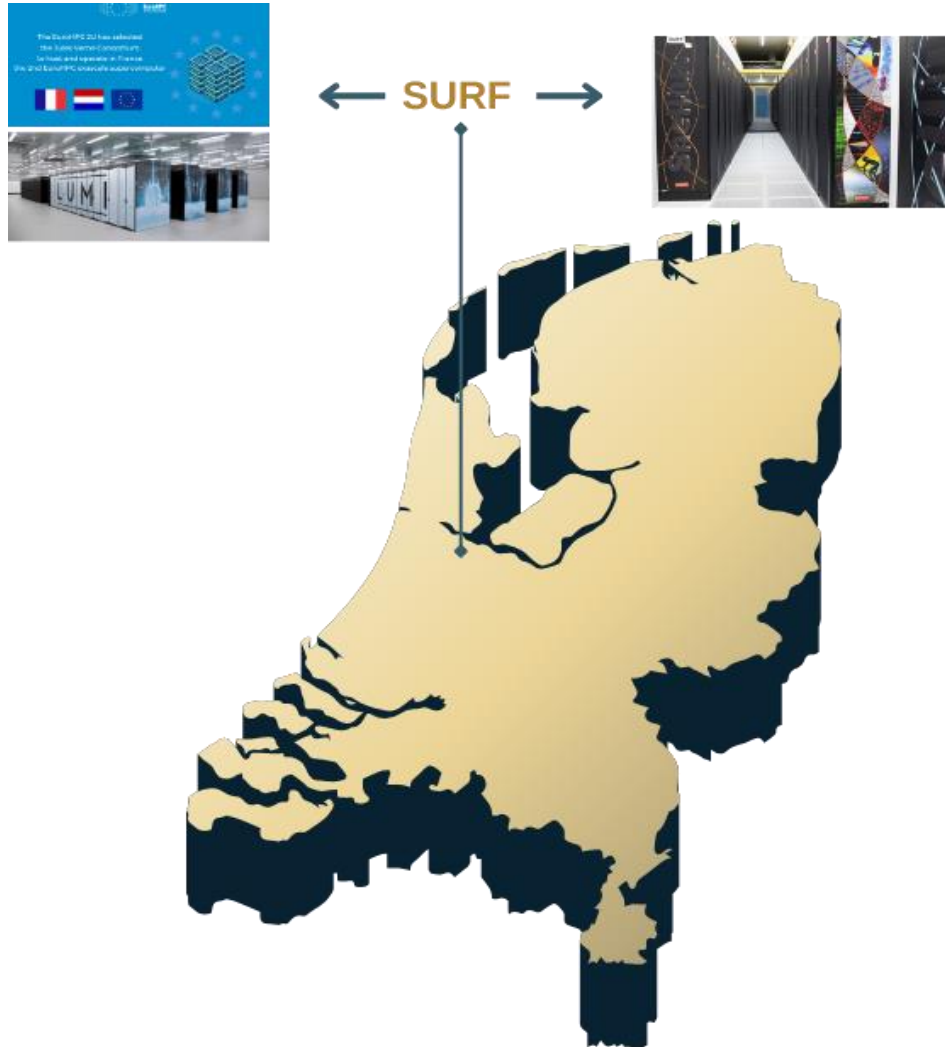
Clícia Pinto and Mohsen Safari
HPC Advisors, SURF

Jun 11th

Amsterdam Science Park



Who we are



SURF Expertise Pool:

- High-Performance Computing
- High-Performance Machine Learning
- High-performance visualization
- Quantum Computing
- Data preservation
- Data management
- Cloud for Research
- System administration
- Trust and Identity
- Software Management

Who we are



← SURF →



- Picture and short bio

Who we are



← SURF →



- Picture and short bio

Outline

- GPU as hardware accelerator: an overview
 - CPU vs GPU
- CUDA programming and execution model
 - PyCUDA programming
 - Numba programming
- Examples:
 - Vector (1D) addition
 - Matrix (2D) addition
 - Matrix (2D) transposition
 - Reduction
 - Matrix (2D) multiplication

Resources

- Slides and source code of the examples can be found at:
 - ???
- Programming environment (jupyter or vscode):
 - <https://ondemand.snellius.surf.nl/>
- To reset your password and Accept the User Agreement:
 - <https://portal.cua.surf.nl>
- To have more information on Snellius partitions and hardware:
 - <https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/30660209/Snellius+partitions+and+accounting>

Implementation via VSCODE / OOD

1. Login to OOD: <https://ondemand.snellius.surf.nl/>

2. Open the shell

3. Install packages:

```
$ module load 2024
```

```
$ module load CUDA/12.6.0
```

```
$ module load Python/3.12.3-GCCcore-13.3.0
```

```
$ pip install pycuda
```

```
$ pip install numba
```

```
$ mkdir gpucourse
```

```
$ cd gpu course
```

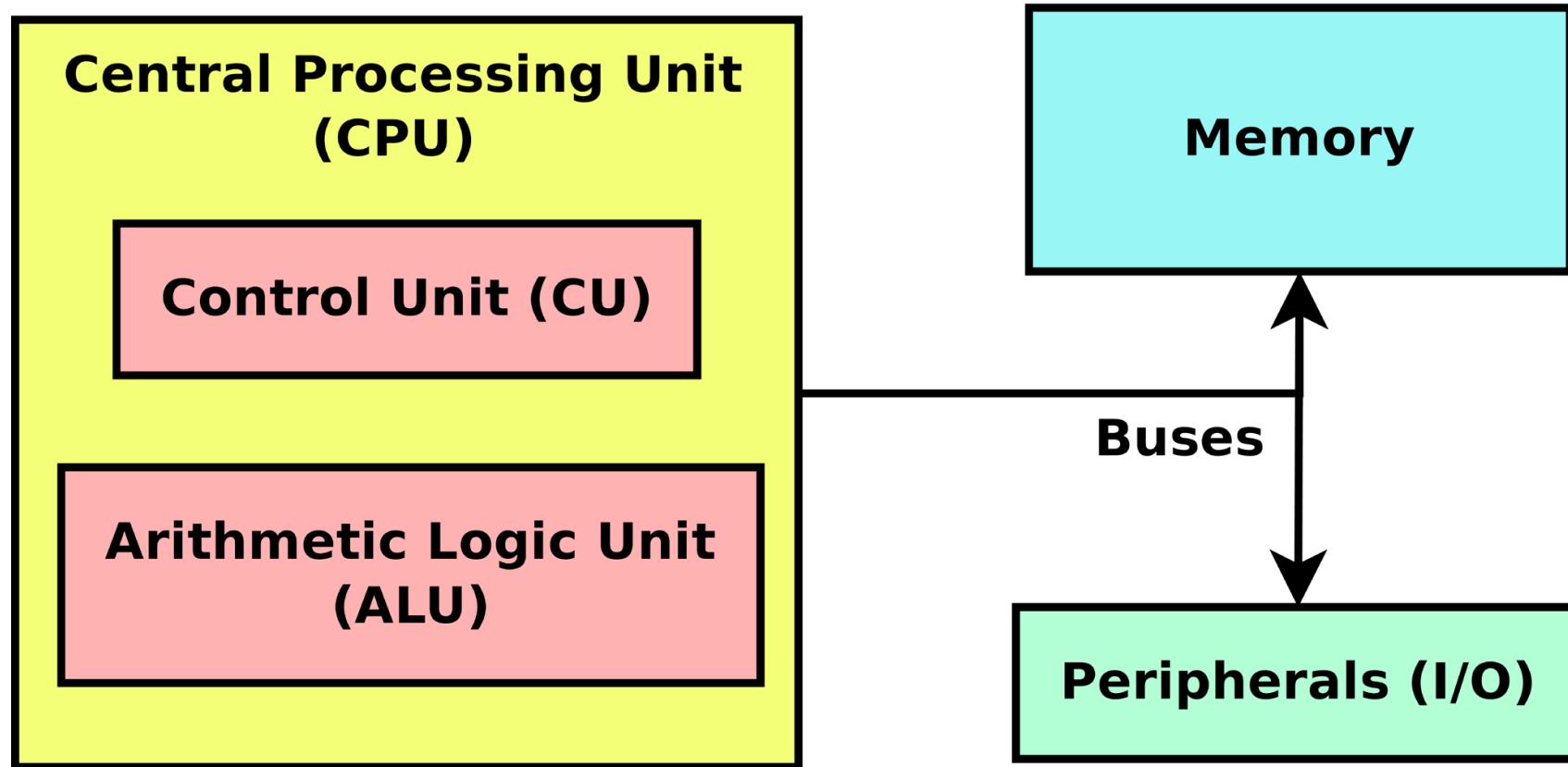
```
$ wget https://github.com/sara-nl/Parallel-and-GPU-programming-in-Python/archive/refs/heads/main.zip
```

```
$ unzip main
```

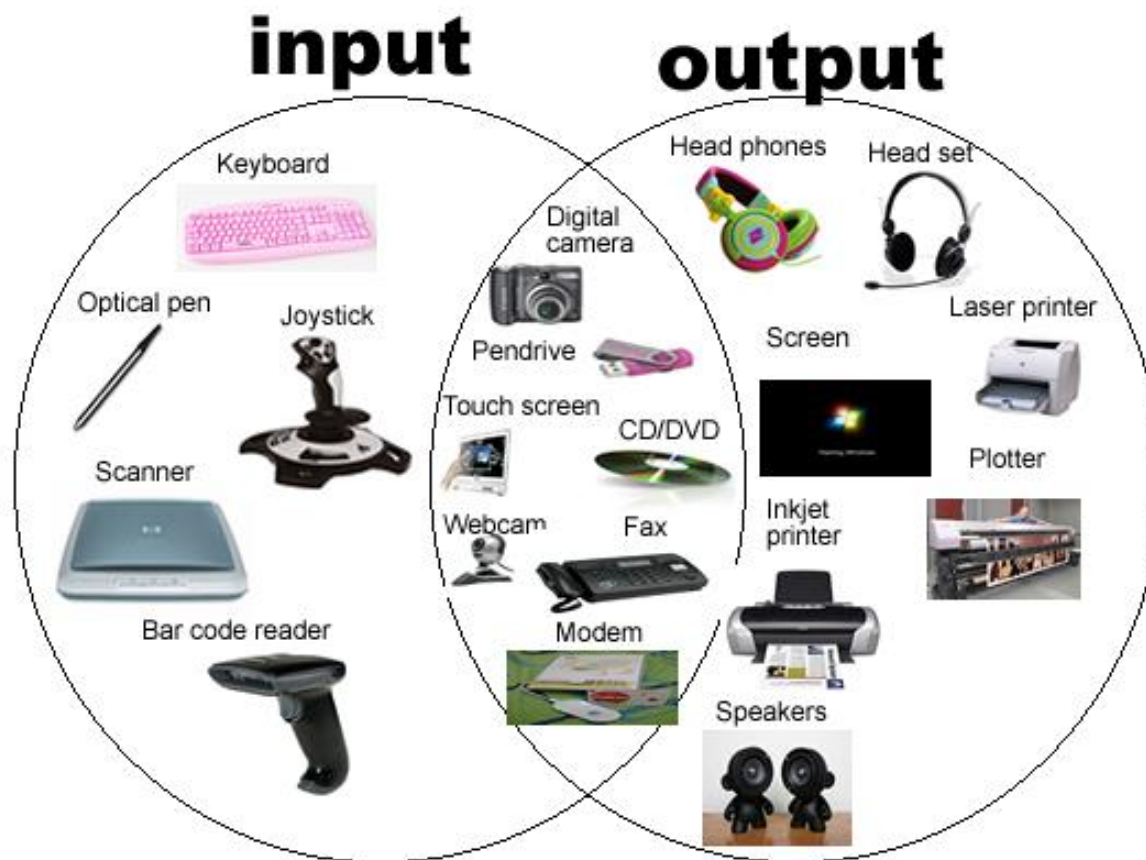

Hardware Accelerator (e.g., GPUs)

- What is it?
- Why do we need it?
- How can we benefit from it?

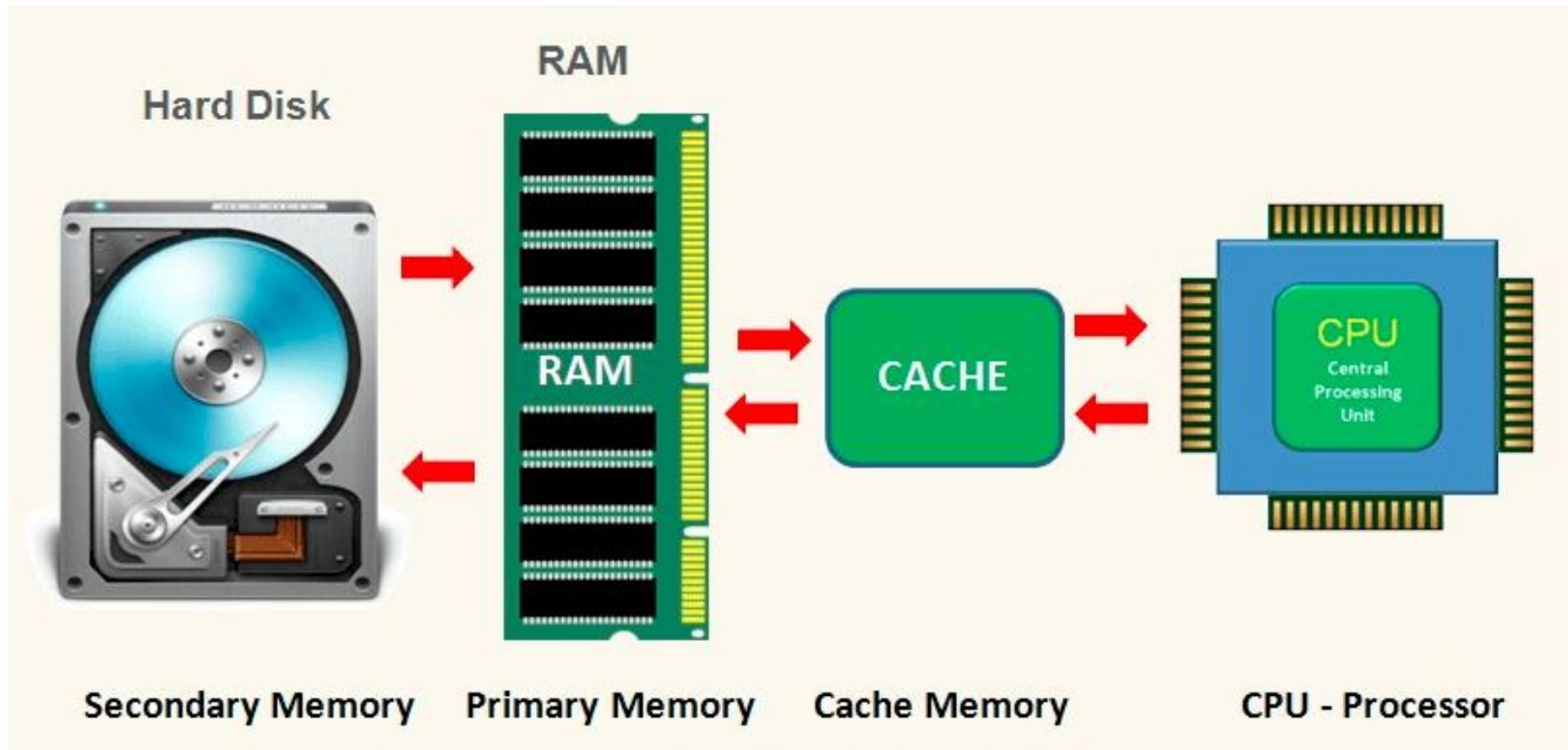
A computer is



Peripherals



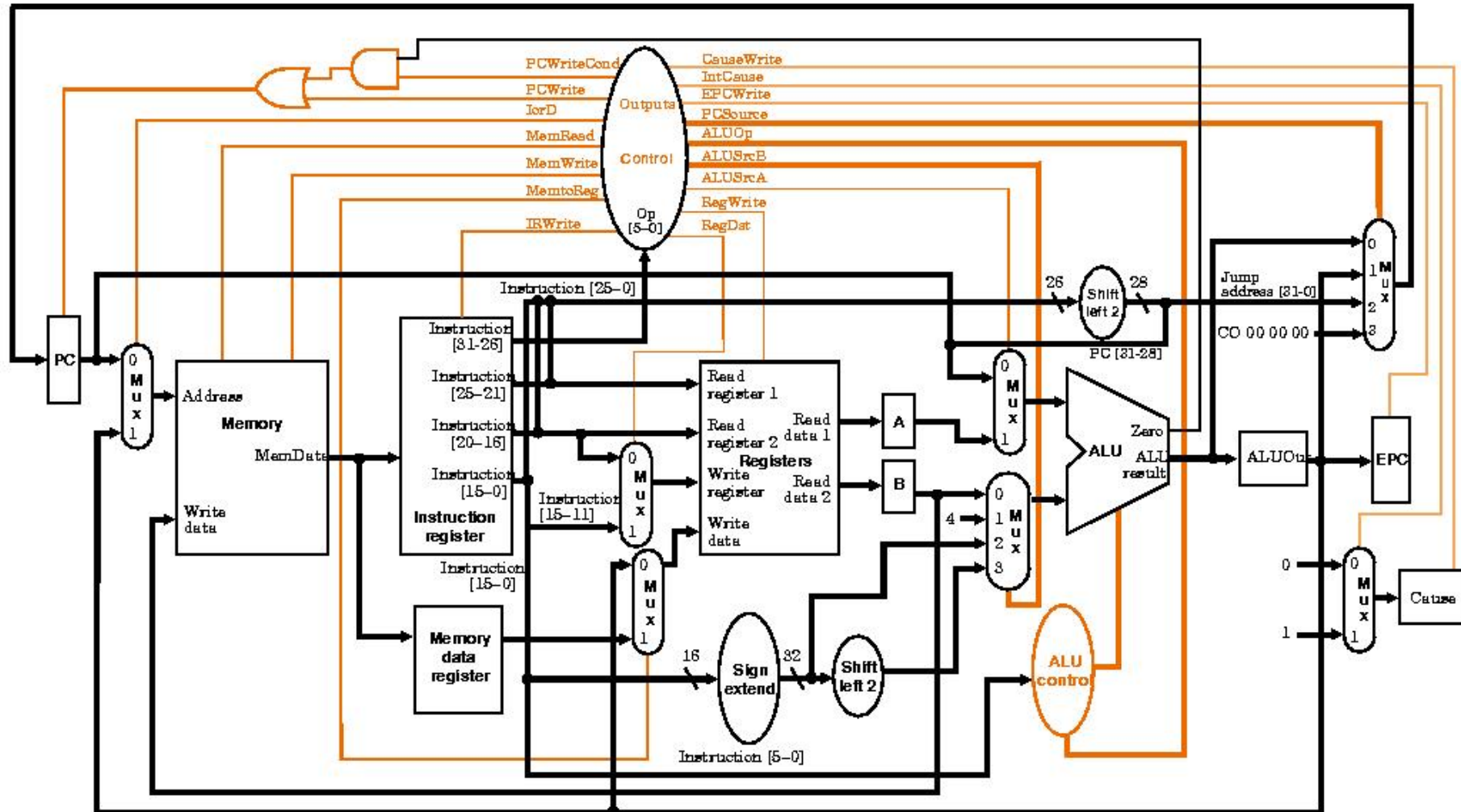
Memory Hierarchy



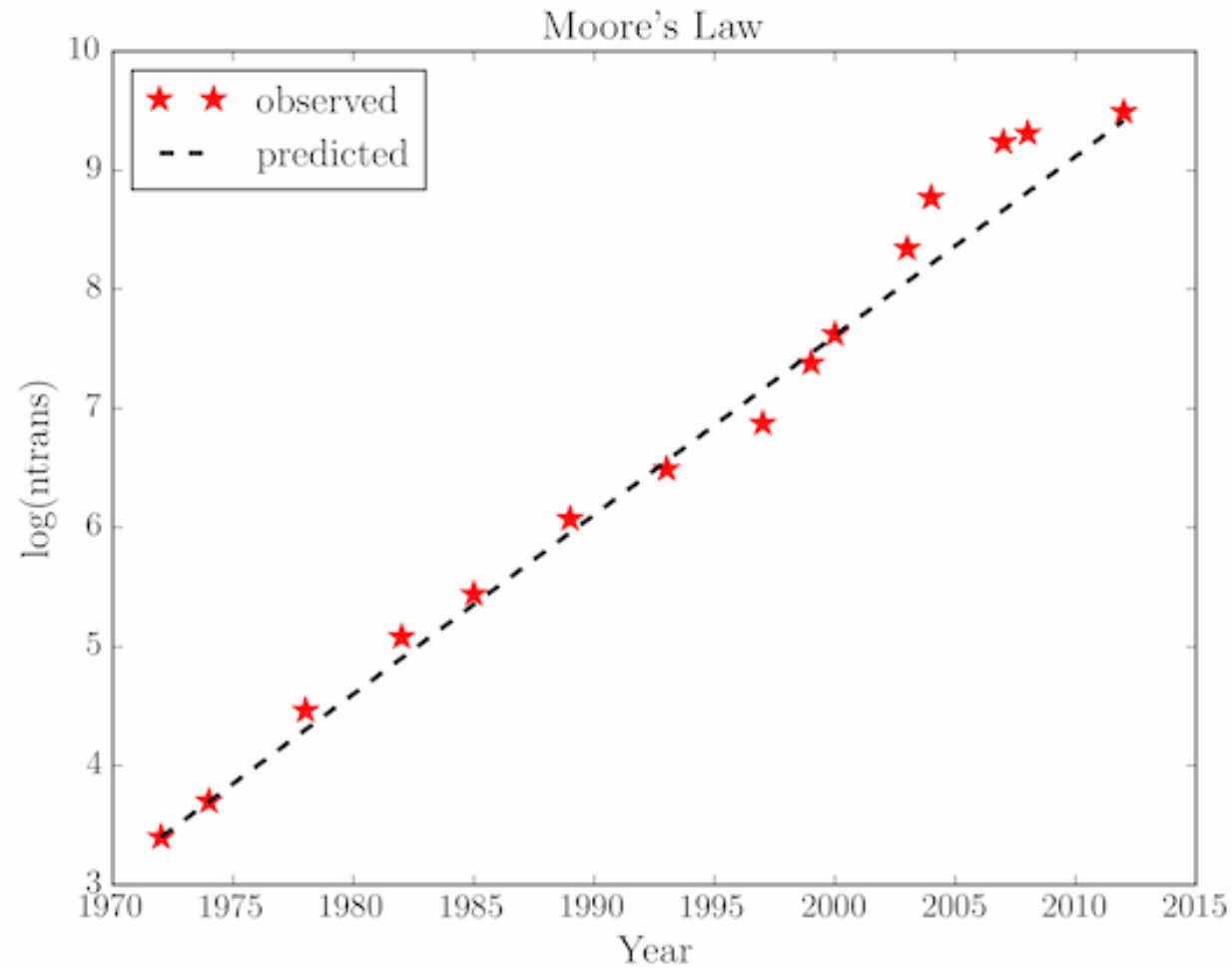
Main Goals

- General-Purpose
- Low latency

Complicated CPUs



Moore's Law

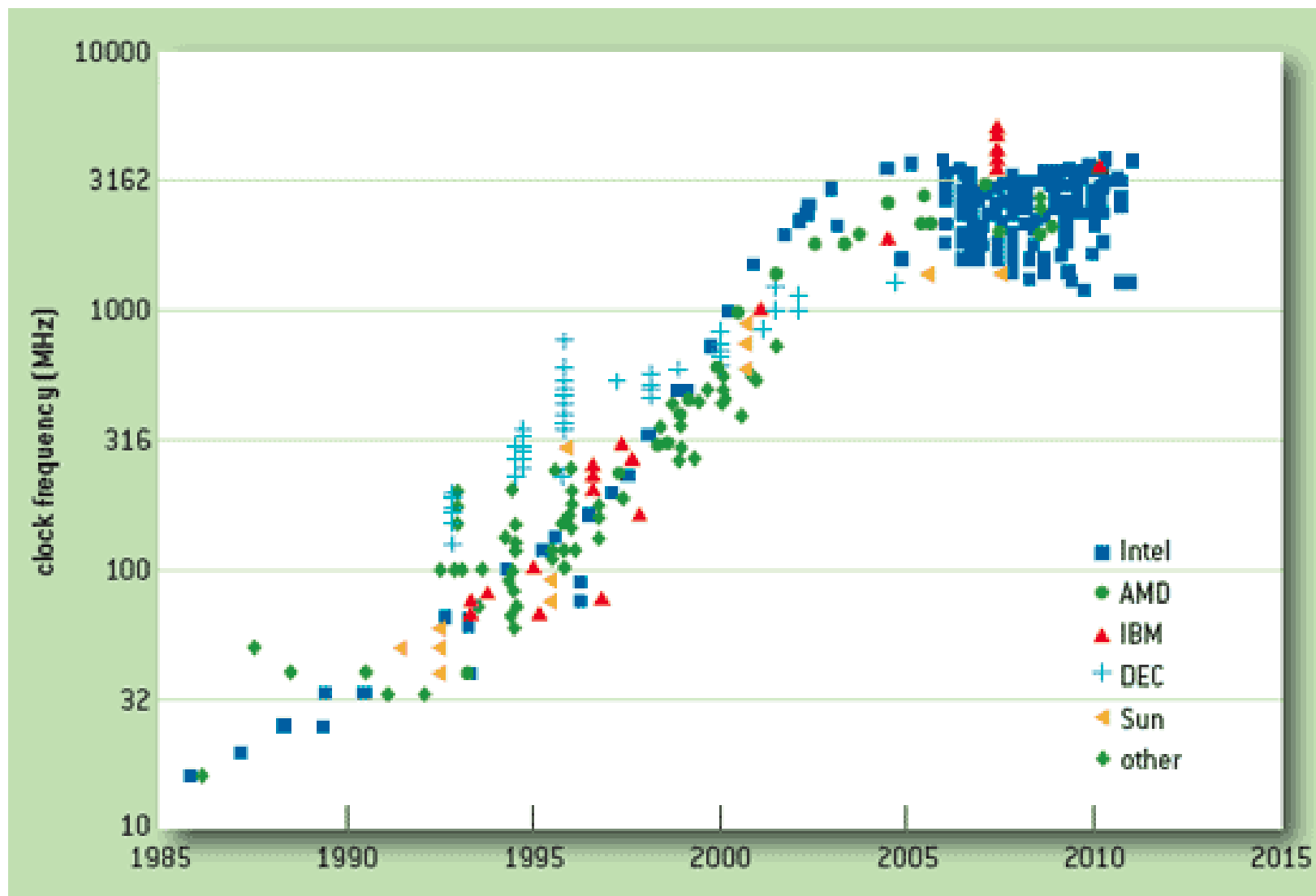


Number of transistors on a CPU chip will double every 18 months

Dennard Scaling Law

- As transistors become smaller, their power density stays constant
- As a result of Moore's and Dennard's law:
 - **CPU manufacturers can raise clock frequency without significantly increasing overall circuit power consumption**

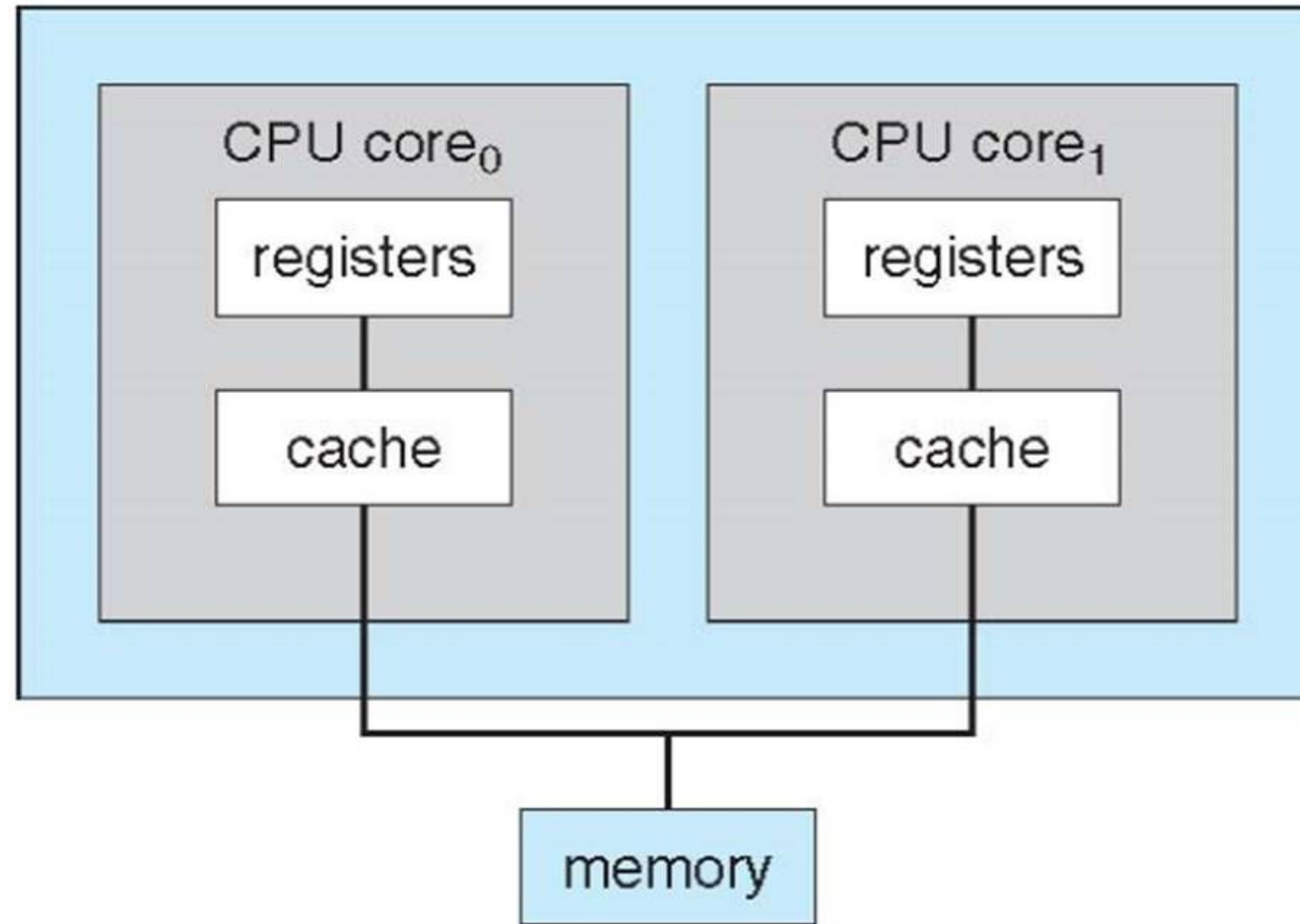
Clock Frequency



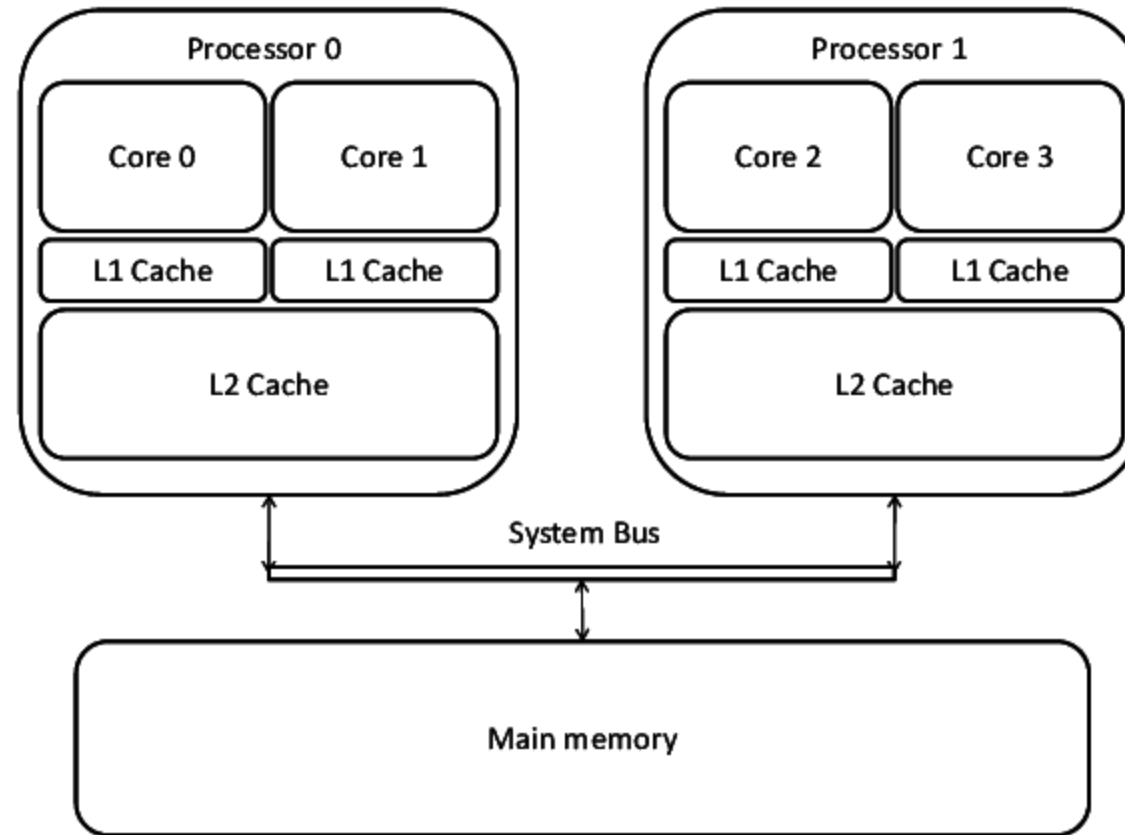
End of Moore's/Dennard's Law

- The prediction had been true for a long time
- We observe that #transistors does not increase in the scale of Moore's law
- We reach the end of Dennard scaling law

Multi-core CPUs



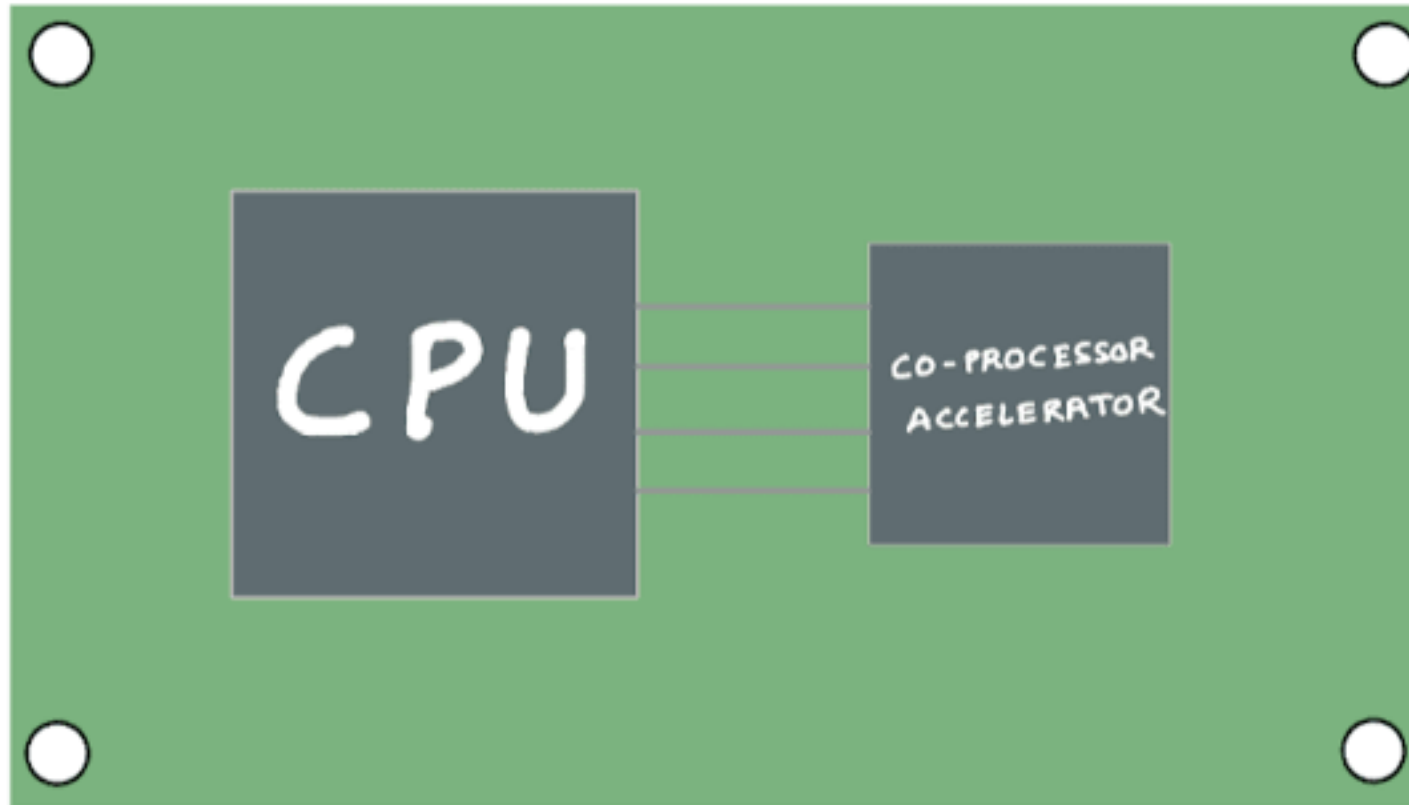
Multi-processors



New requirements

- Big data
- AI
- New applications:
 - Massively parallel
 - Specific operations
- Faster computation

Accelerator



Accelerators/Co-processors

- Graphics Processing Units (GPUs)
- Field Programmable Gate Arrays (FPGAs)
- Tensor Processing Units (TPUs)

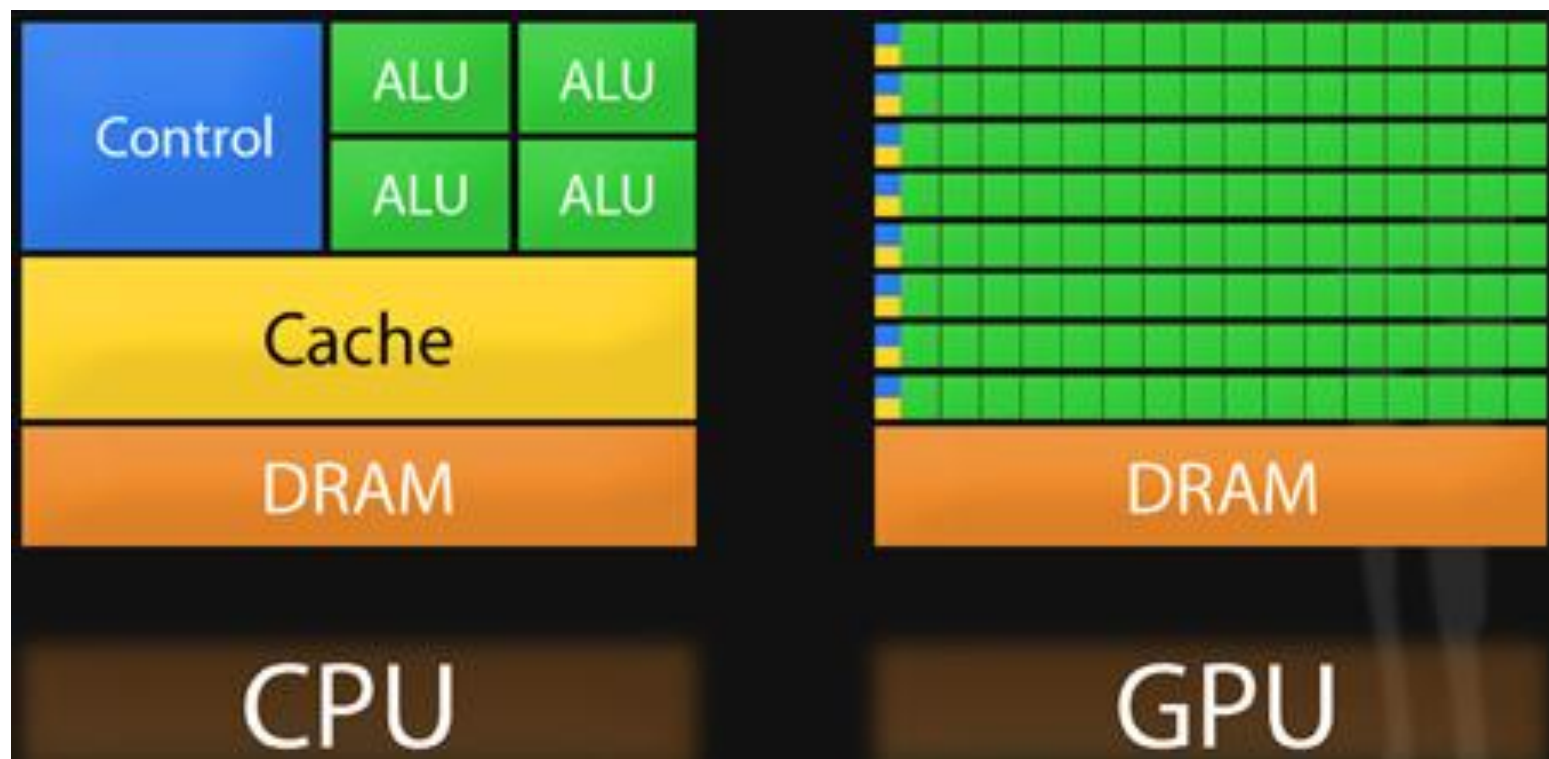
Simpler many cores

- Simpler cores (i.e., simplified ALUs and CUs)
- Replicate many of them
- As a co-processor

GPUs

- Initially invented for image rendering purposes
- Gradually evolved to be used as General Purpose GPU

GPUs vs CPUs



Two Metrics

- **Latency:**
 - The time it takes an instruction to be processed
- **Throughput:**
 - The number of instructions that can be processed in a certain amount of time

GPU Manufacturers



Supercomputers

Top 500 list

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure United States	2,073,600	561.20	846.84	

Supercomputers

Top 500 list

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
6	HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461
7	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
8	Alps - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS, HPE Swiss National Supercomputing Centre (CSCS) Switzerland	2,121,600	434.90	574.84	7,124
9	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.70	531.51	7,107
10	Leonardo - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, EVIDEN EuroHPC/CINECA Italy	1,824,768	241.20	306.31	7,494

Supercomputers

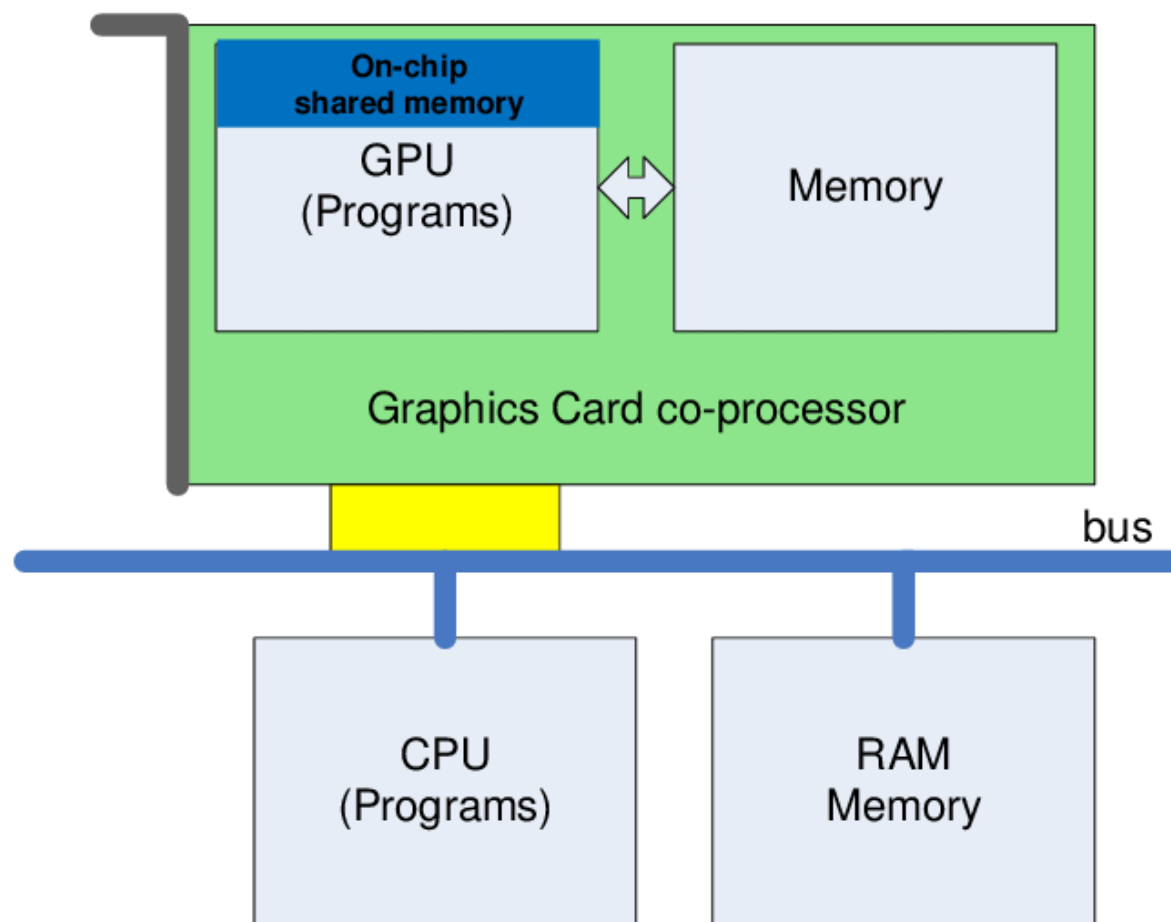
Top 500 list

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
112	Snellius Phase 3 GPU - ThinkSystem SD665-N V3, AMD EPYC 9334 32C 2.7GHz, Nvidia H100 SXM5 94Gb, Infiniband HDR200, RHEL 9.2, Lenovo SURF Netherlands	52,096	13.64	21.66	296

Snellius Supercomputer

- 72 GPU nodes with 4 A100 NVIDIA GPUs; 288 A100 GPUs
- 88 GPU nodes with 4 H100 NVIDIA GPUs; 352 H100 GPUs
- In total 640 GPUs

GPU CPU Connectivity

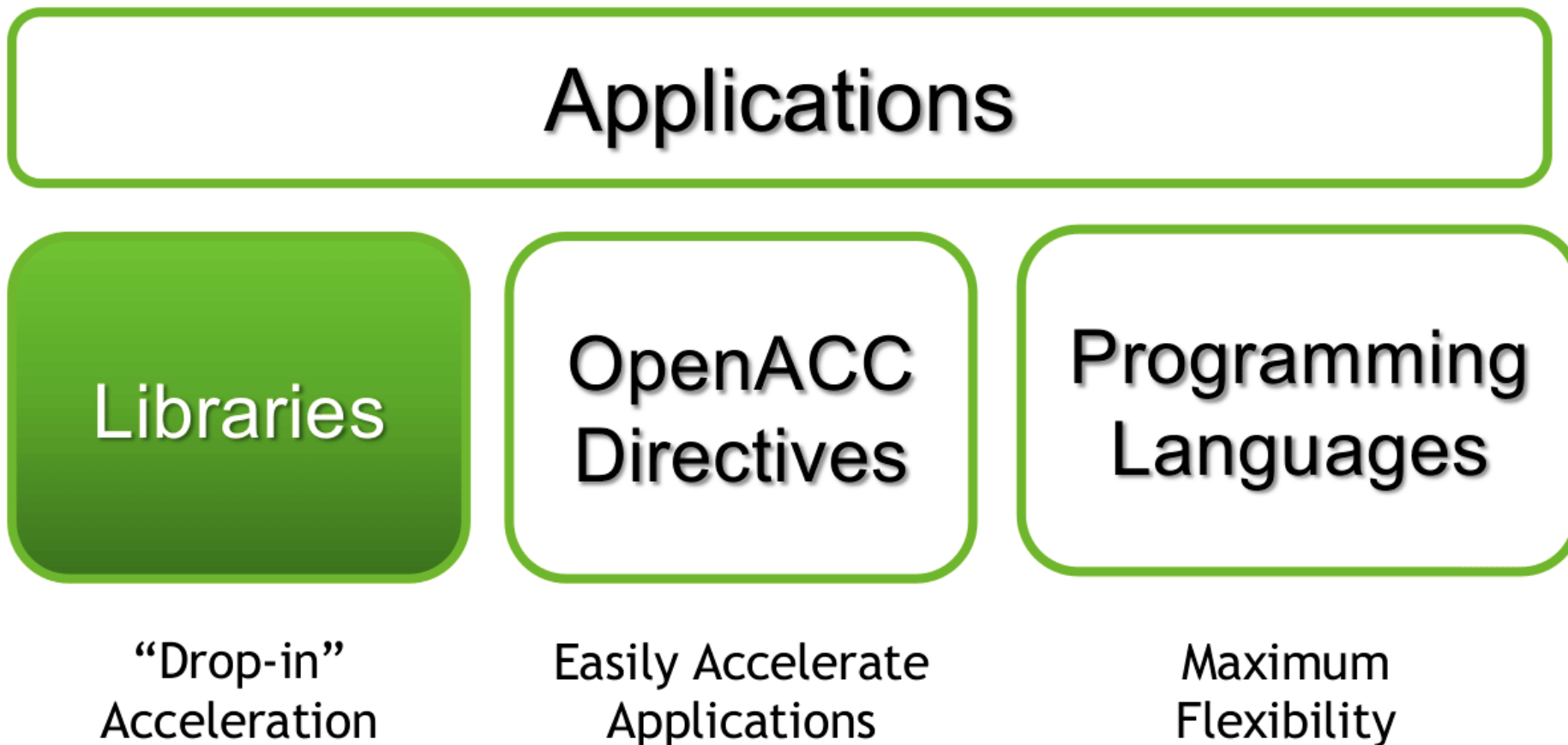


GPU Usability

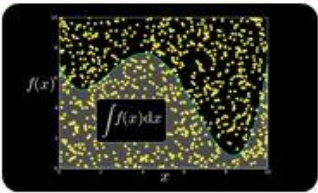
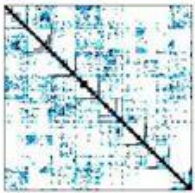
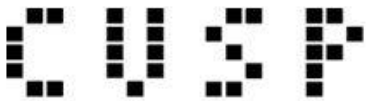
- How to Use GPUs?

GPU Usability

3 Ways to Accelerate Applications



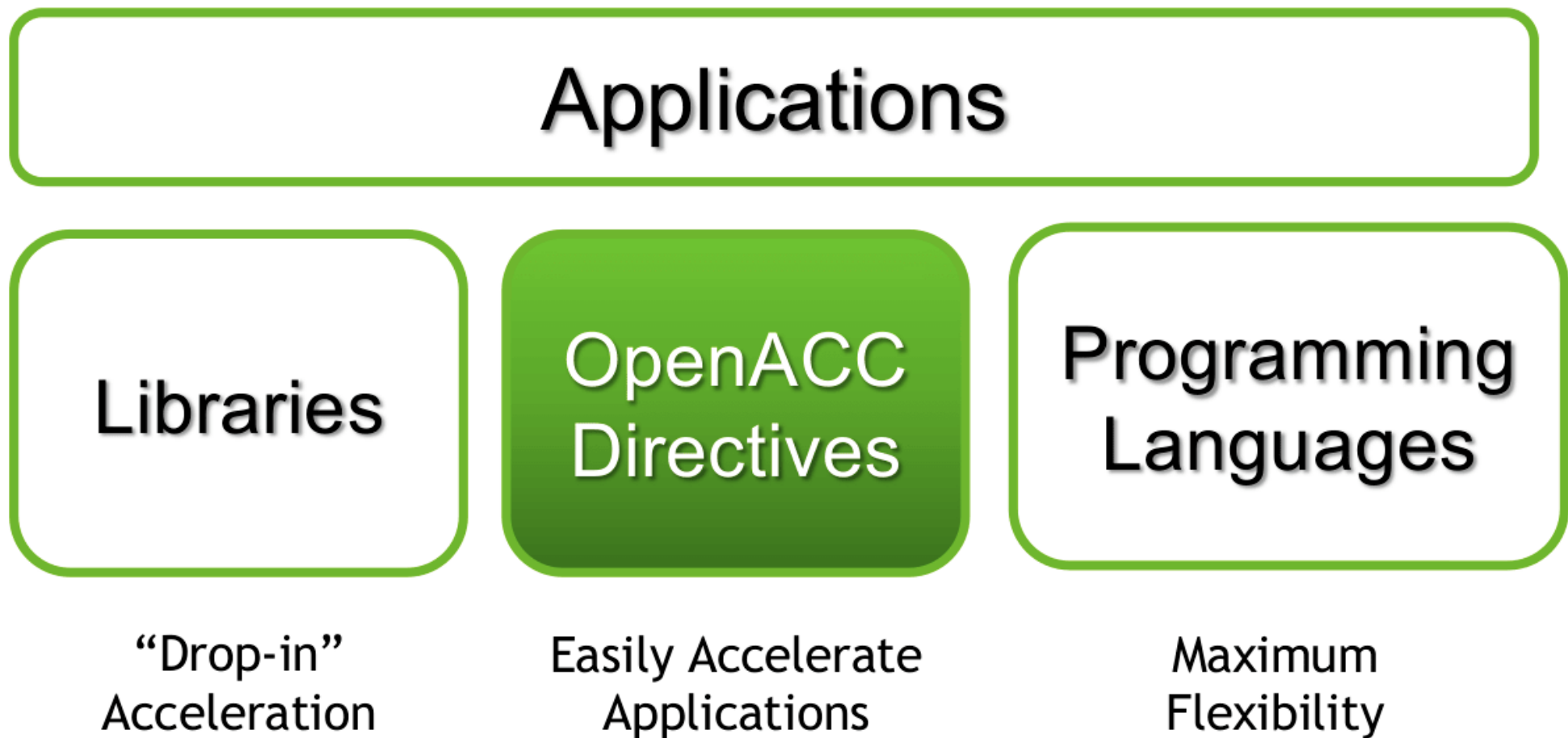
GPU Libraries

 NVIDIA cuBLAS	 NVIDIA cuRAND	 NVIDIA cuSPARSE	 NVIDIA NPP
 Vector Signal Image Processing	 GPU Accelerated Linear Algebra	 Matrix Algebra on GPU and Multicore 	 NVIDIA cuFFT
 IMSL Library	 Building-block Algorithms for CUDA	 Sparse Linear Algebra 	 C++ STL Features for CUDA 

GPU Accelerated Libraries
“Drop-in” Acceleration for Your Applications

GPU Usability

3 Ways to Accelerate Applications



OpenACC/OpenMP

- OpenACC stands for Open Accelerators
- OpenMP stands for Open Multi-Processing
- Directive-based APIs
- Simple compiler hints to parallelize the code

OpenACC/OpenMP

```
#include <stdio.h>
#include <omp.h>

int main() {
    const int N = 1000;
    float a[N], b[N], c[N];

    // Initialize
    for (int i = 0; i < N; i++) {
        a[i] = i;
        b[i] = 2*i;
    }

    // Offload computation to GPU
    #pragma omp target teams distribute parallel for map(to:a[0:N], b[0:N]) map(from:c[0:N])
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }

    // Print a few results
    for (int i = 0; i < 5; i++) {
        printf("%f ", c[i]);
    }
    printf("\n");

    return 0;
}
```

```
#include <stdio.h>

int main() {
    const int N = 1000;
    float a[N], b[N], c[N];

    // Initialize
    for (int i = 0; i < N; i++) {
        a[i] = i;
        b[i] = 2*i;
    }

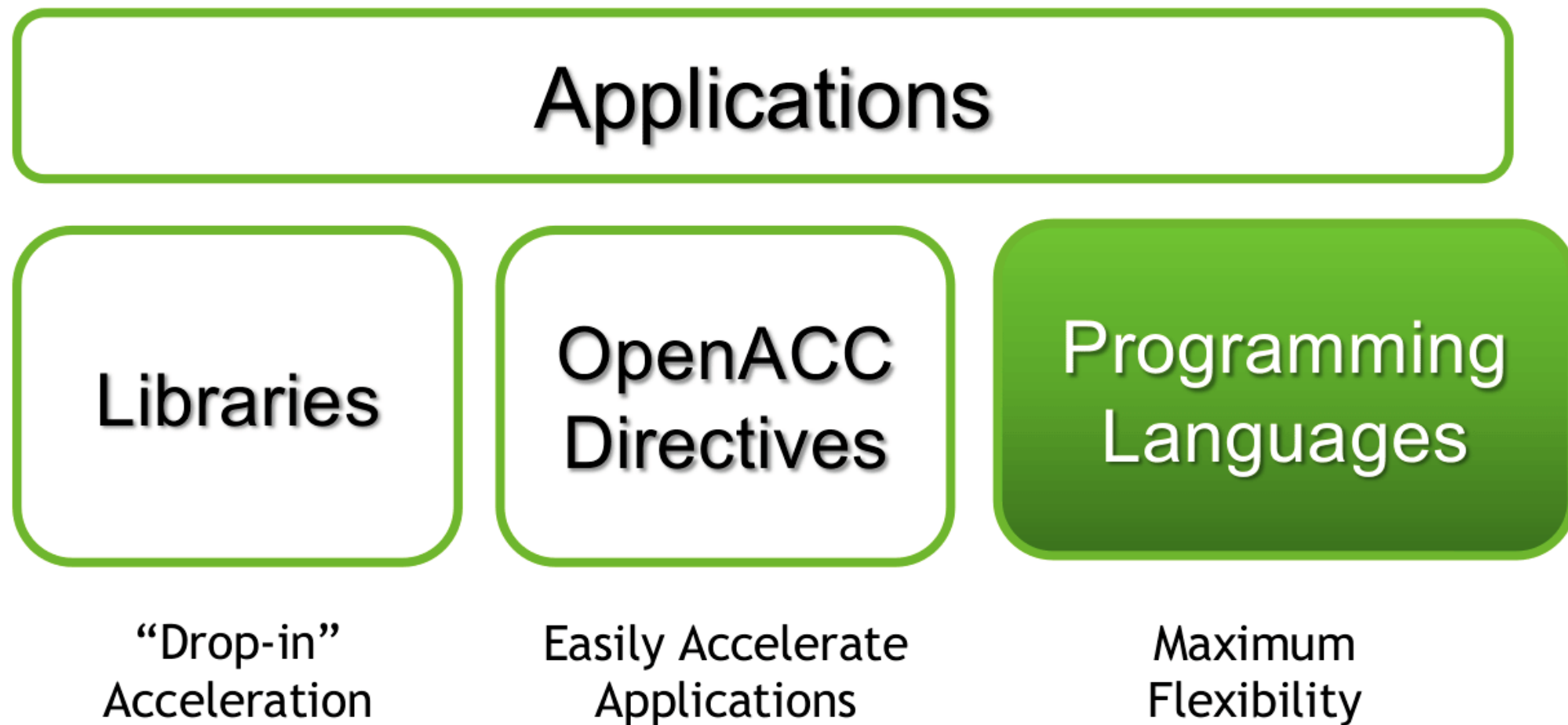
    // Offload computation to the GPU using OpenACC
    #pragma acc parallel loop copyin(a[0:N], b[0:N]) copyout(c[0:N])
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }

    // Print a few results
    for (int i = 0; i < 5; i++) {
        printf("%f ", c[i]);
    }
    printf("\n");

    return 0;
}
```

GPU Usability

3 Ways to Accelerate Applications



GPU Programming Languages

Numerical analytics ▶

MATLAB, Mathematica, LabVIEW

Fortran ▶

CUDA Fortran, OpenACC,
OpenMP4.5

C ▶

CUDA C, OpenCL, OpenACC,
OpenMP4.5

C++ ▶

CUDA C++, Thrust, OpenCL,
OpenACC/OpenMP4.5

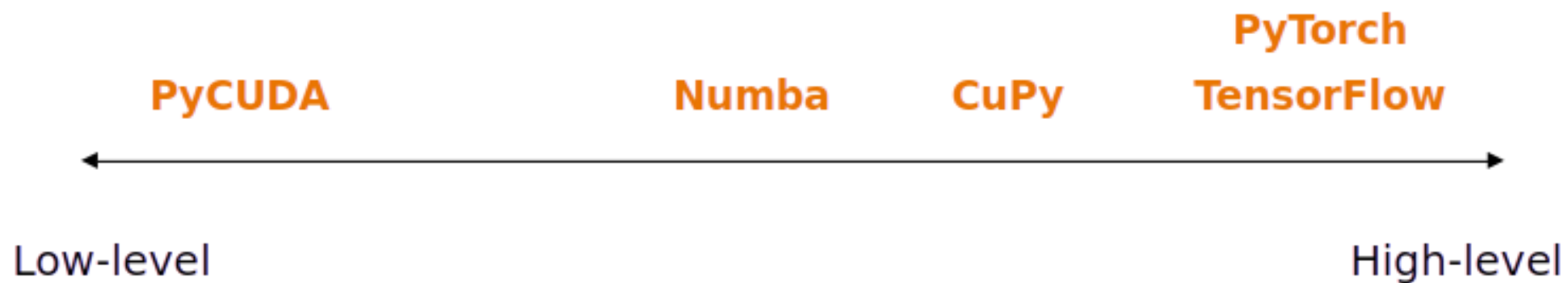
Python ▶

PyCUDA/PyOpenCL, Numba, ...

Core GPU Programming

- **Nvidia GPUs:**
 - CUDA, OpenCL, HIP
- **AMD GPUs:**
 - OpenCL, HIP
- **Intel GPUs:**
 - OpenCL, SYCL - oneAPI DPC++

Accessing to GPUs in Python



PyTorch/TensorFlow

- They are powerful and mature deep learning libraries
- They benefit from GPUs without knowing GPU programming knowledge
- They are open sources
- They are taught in machine learning courses

CuPy vs NumPy



```
import numpy as np
X_cpu = np.zeros((10,))
W_cpu = np.zeros((10, 5))
y_cpu = np.dot(x_cpu, W_cpu)
```



```
import cupy as cp
x_gpu = cp.zeros((10,))
W_gpu = cp.zeros((10, 5))
y_gpu = cp.dot(x_gpu, W_gpu)
```

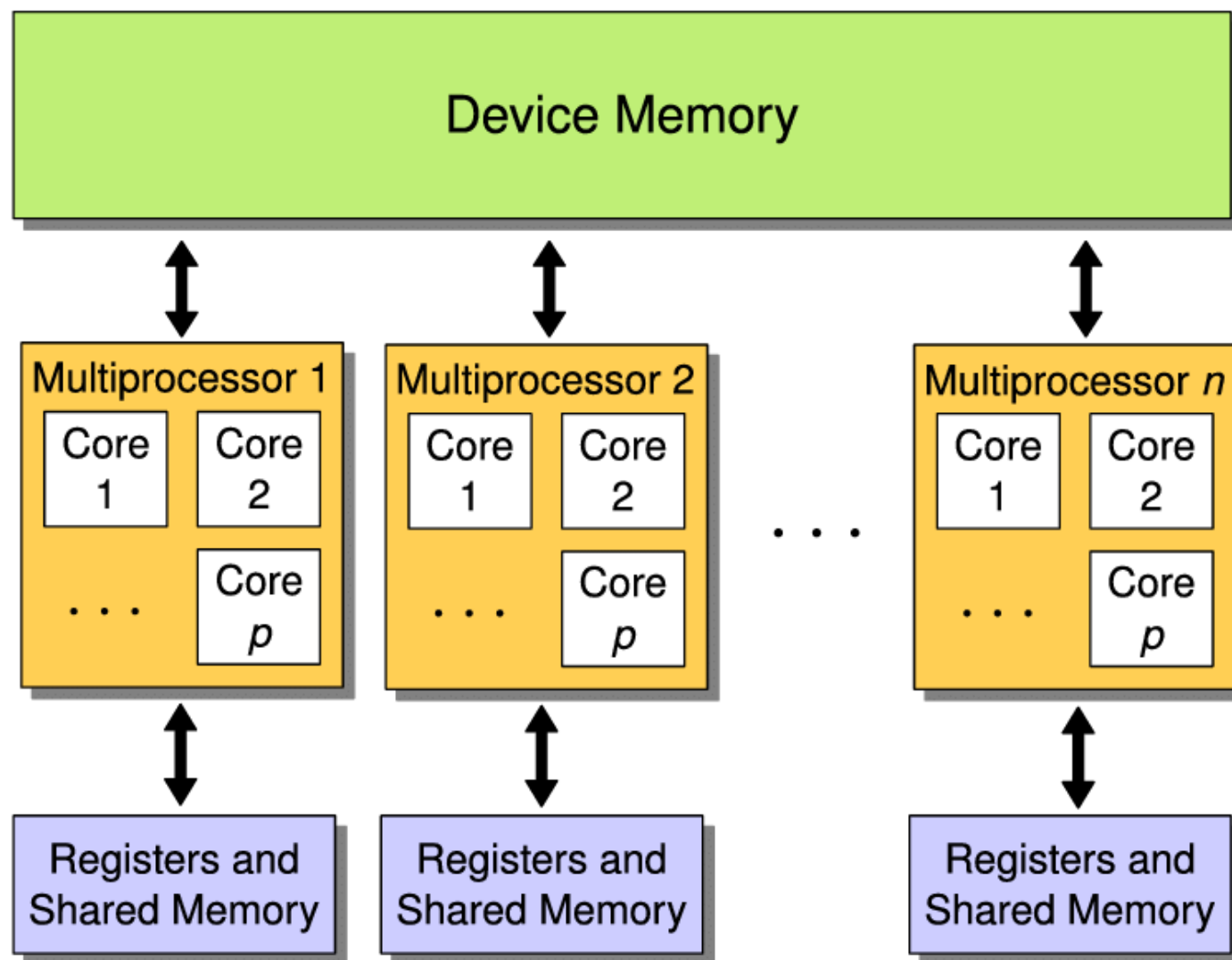
Numba

- It is an open-source Just-In Time (JIT) compiler that translates a subset of Python and Numpy into GPU machine code
- It uses a collection of decorators that can be applied to your functions to instruct Numba to compile them
- For more information: <https://numba.pydata.org/>

PyCUDA

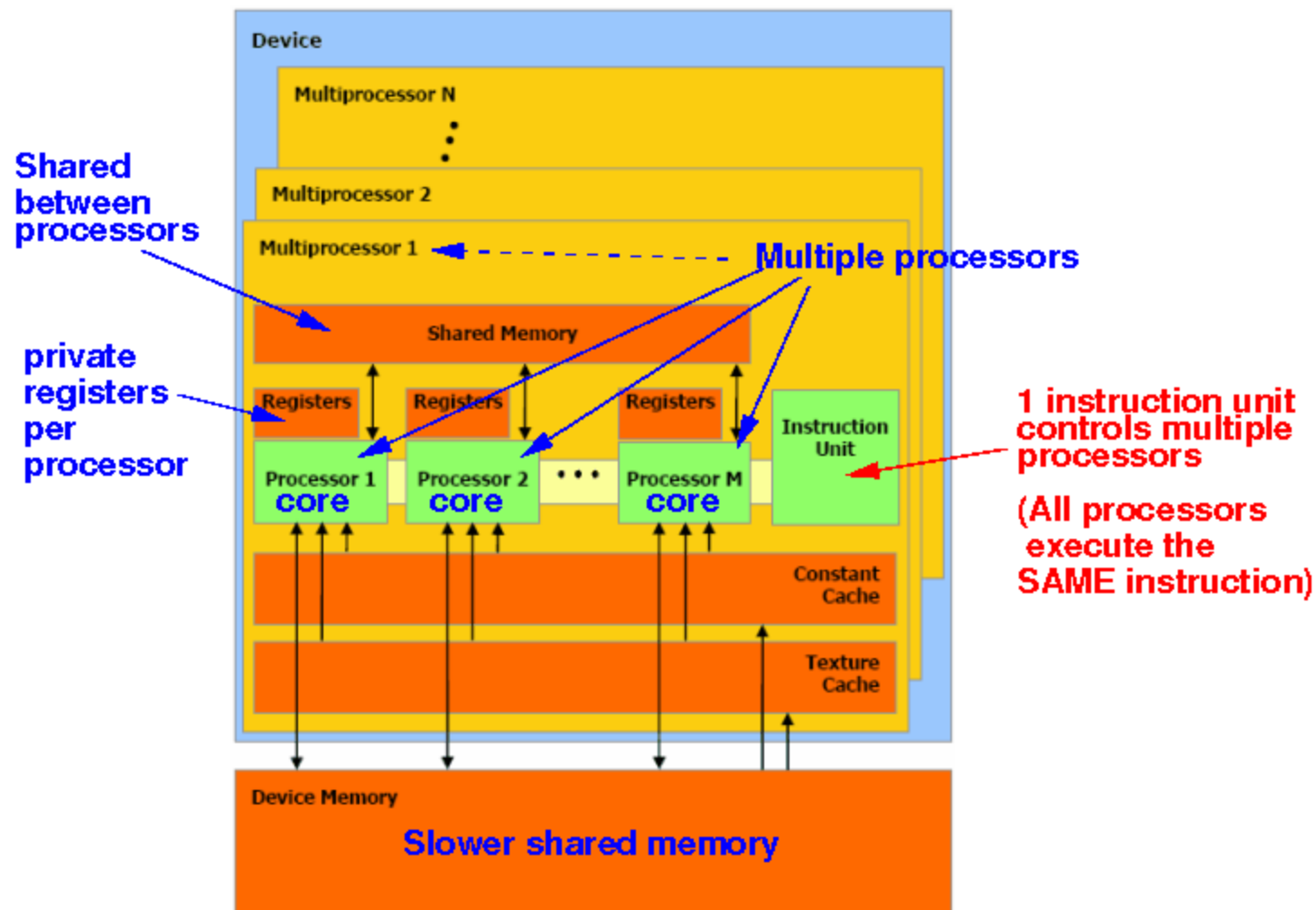
- It gives you easy, Pythonic access to NVIDIA's CUDA parallel computation API
- There is more flexibility to write custom CUDA kernels
- For more information: <https://documen.tician.de/pycuda/>

NVIDIA GPU Hardware



NVIDIA GPU Hardware

GPU device:



Flynn's classical taxonomy

		Instruction stream	
		Single	Multiple
Data stream	Single	SISD	MISD
	Multiple	SIMD	MIMD

CUDA Programming Model

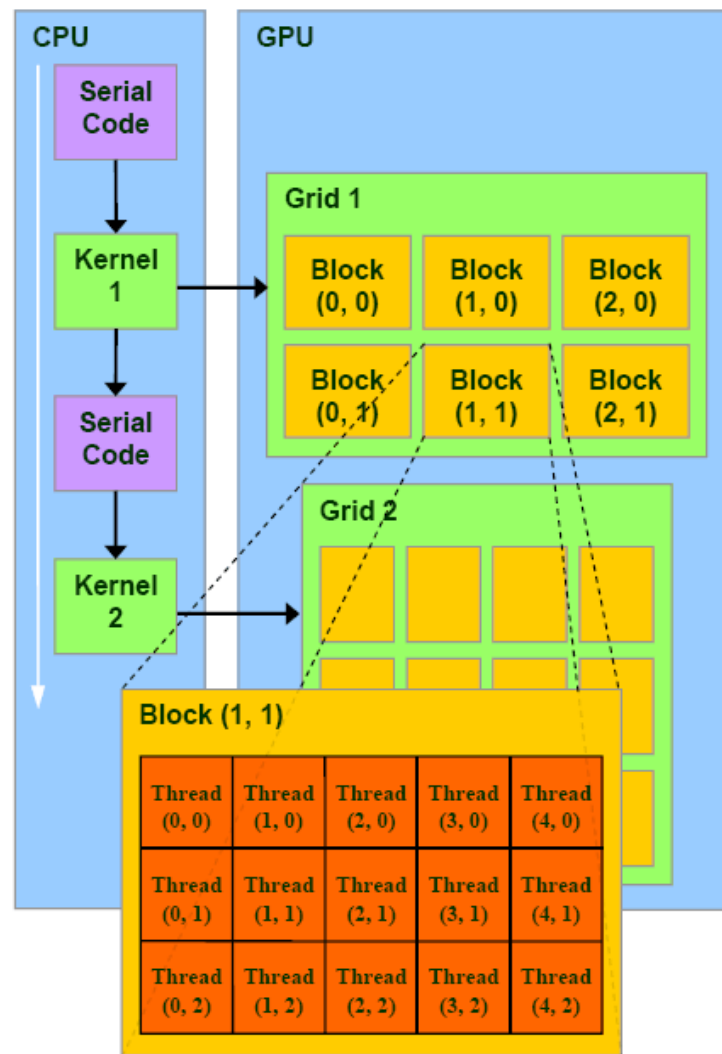
- Introduced by NVIDIA in 2006, Compute Unified Device Architecture
- General purpose programming model that leverages the parallel compute engine in NVIDIA GPUs
- An extension of C language
- CUDA programs are CPU-GPU programs:
 - CPU part is called host
 - GPU part is called kernel

CUDA Programming Model

To execute any CUDA program, there are three main steps:

1. Copy the input data from host memory to device memory, also known as host-to-device transfer
2. Call the kernel from host and execute the GPU program
3. Copy the results from device memory to host memory, also called device-to-host transfer

CUDA Programming Model



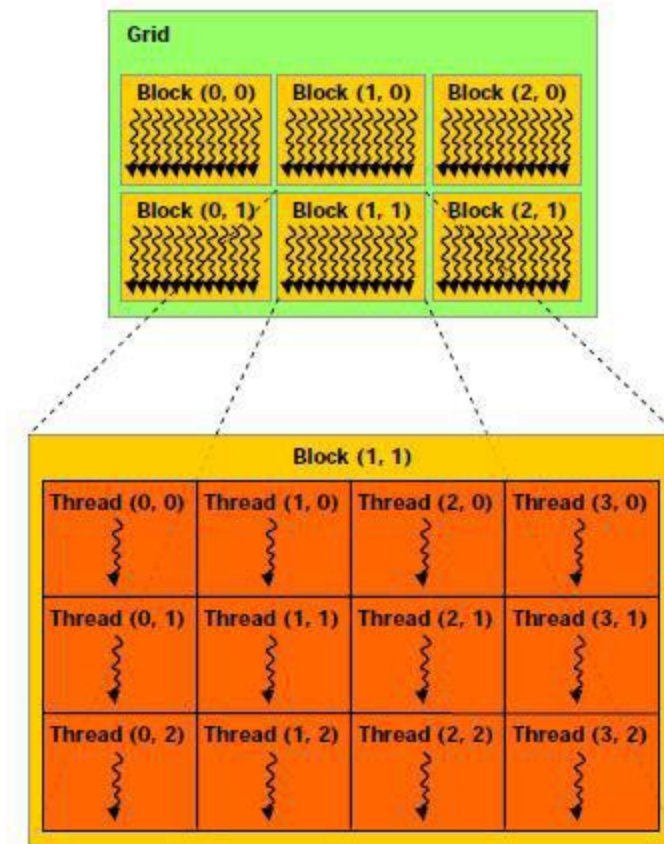
CUDA Programming Model

Threads are organized into two hierarchical levels:

- Threads are grouped into blocks
- Blocks are grouped into grids

Blocks and grids can be

- 1D, 2D and 3D



CUDA Programming Model

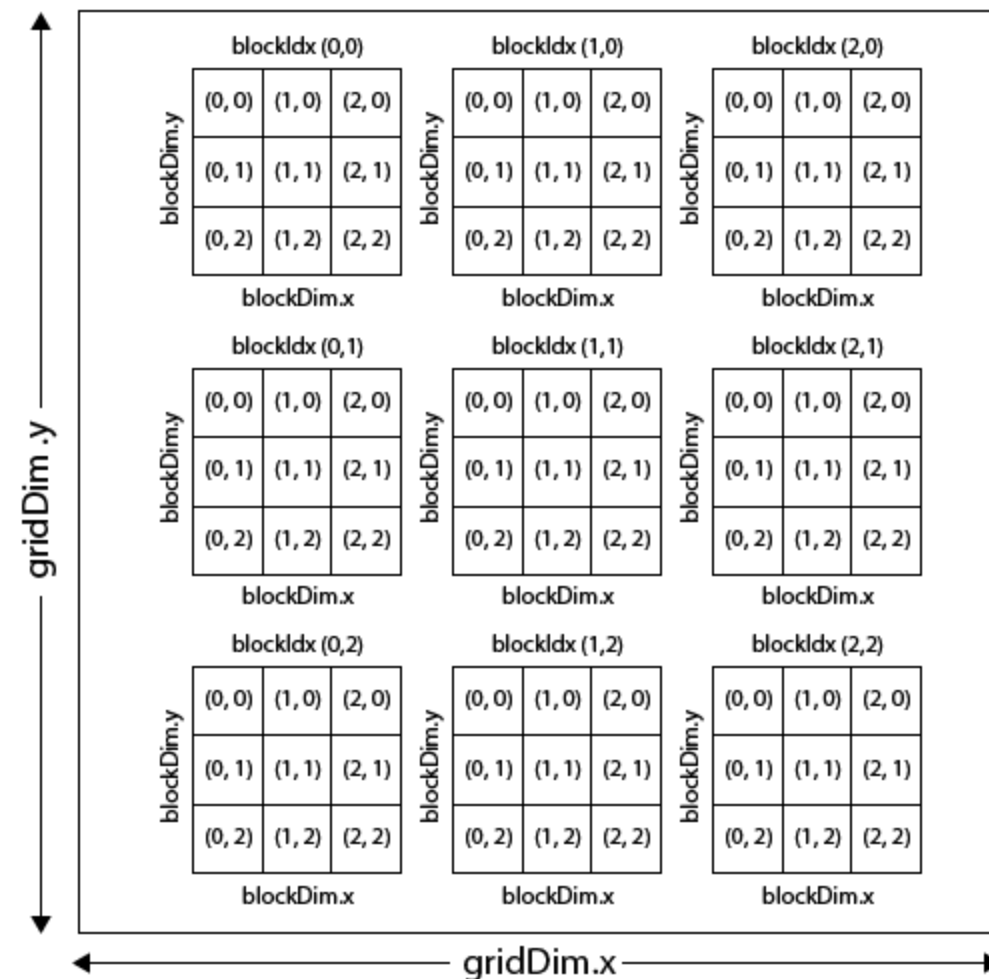
Built-in functions:

- **Dimension:**
 - `gridDim.x`, `gridDim.y`, `gridDim.z`
 - `blockDim.x`, `blockDim.y`, `blockDim.z`
- **Index:**
 - `blockIdx.x`, `blockIdx.y`, `blockIdx.z`
 - `threadIdx.x`, `threadIdx.y`, `threadIdx.z`

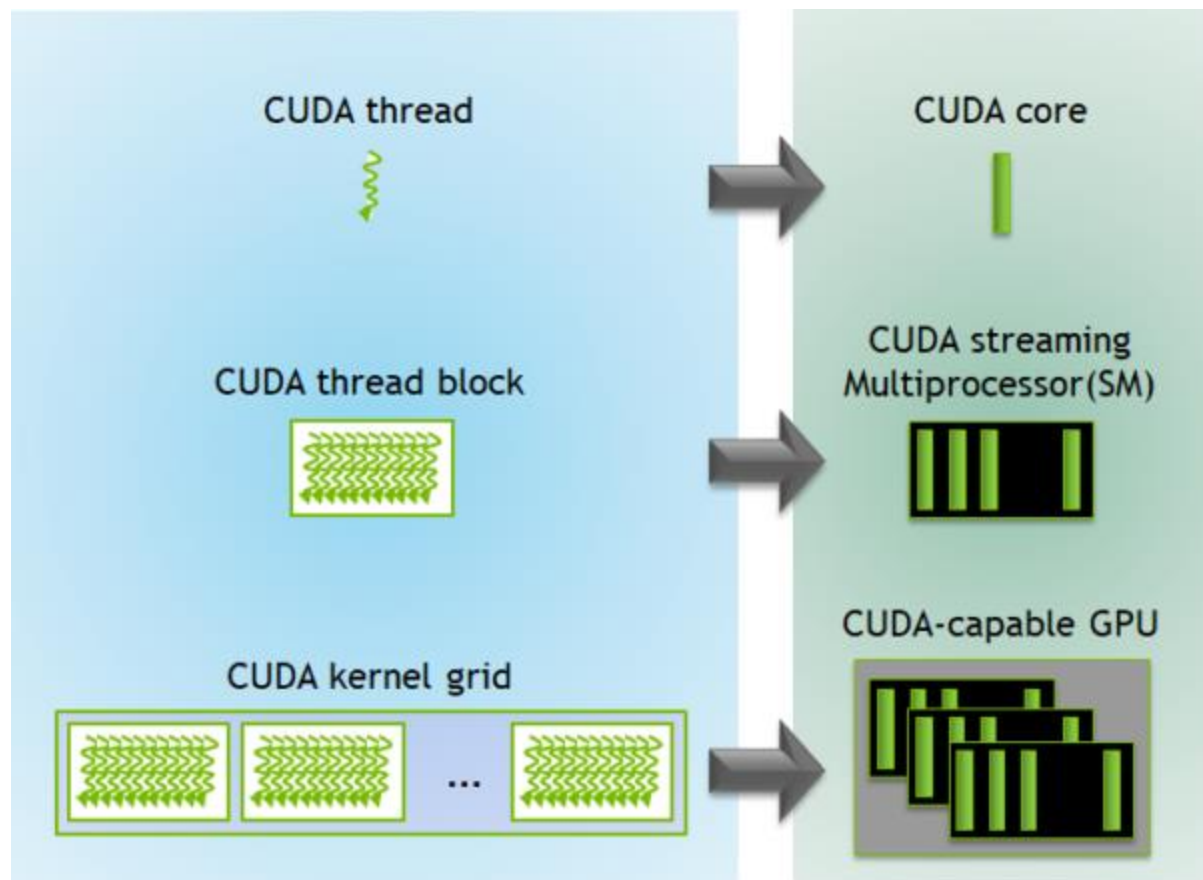
CUDA Programming Model

- `gridDim.x = 3`
- `gridDim.y = 3`
- `blockDim.x = 3`
- `blockDim.y = 3`

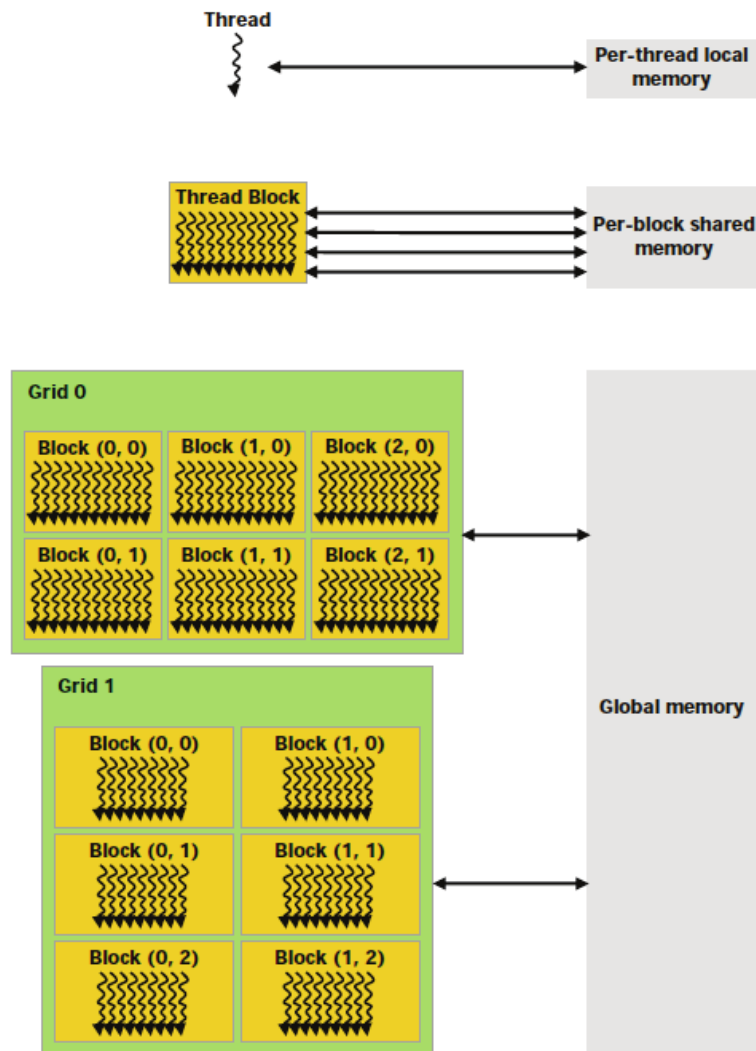
CUDA Grid



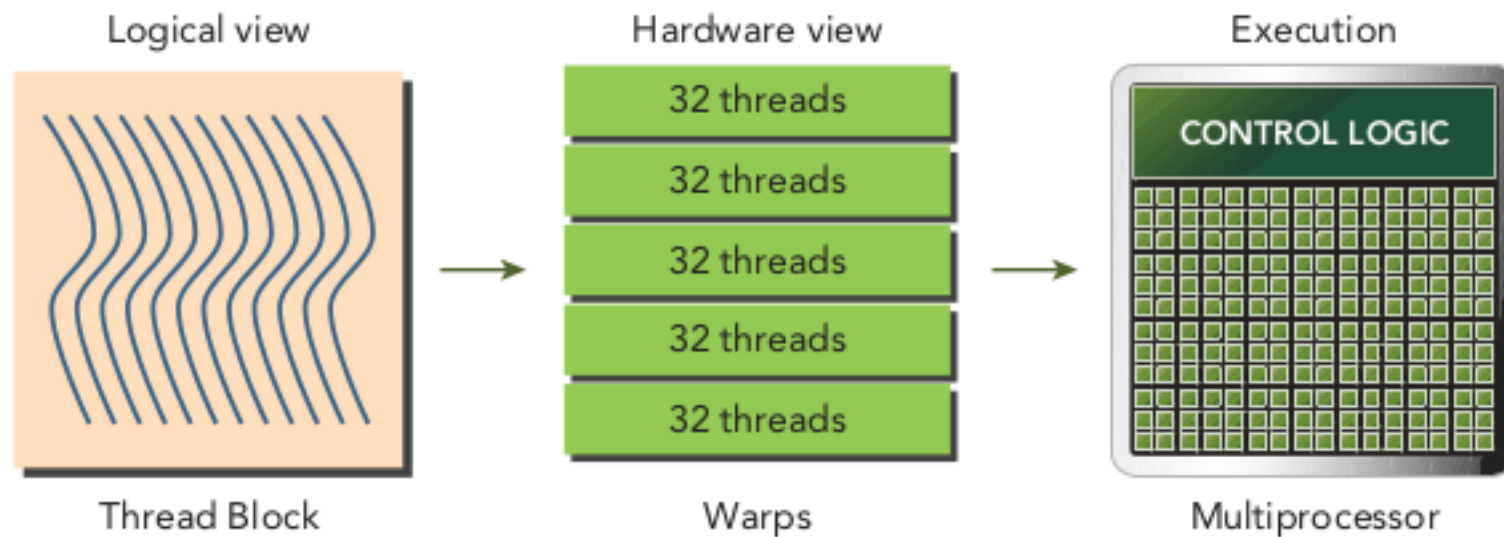
CUDA Execution Model



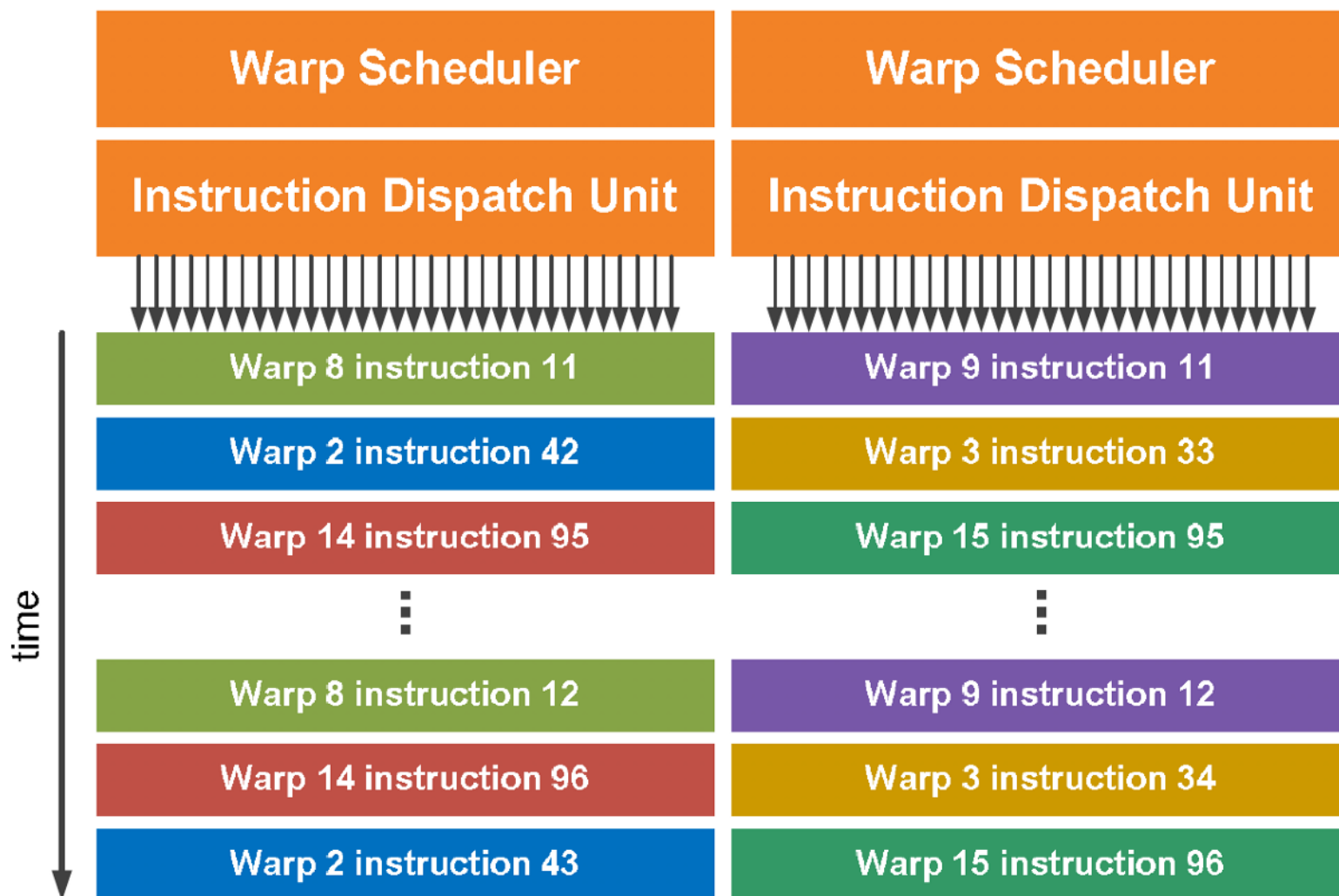
CUDA Execution Model



CUDA Execution Model



CUDA Execution Model



Synchronization in CUDA

- There is a mechanism to synchronize all threads in a block:
 - Built-in function `__syncthreads()`
- There is no mechanism to synchronize all threads across all blocks
 - Decouple the kernel into two separate kernels

GPU Node

- 4 NVIDIA A100 GPUs per node
 - Multiprocessors: 108
 - Streaming cores: 6912
 - Tensor Cores: 432
 - Global memory: 40 GB
- MIG partitions: 1/7th of A100 GPUs
- One GPU is shared among 7 people
- Note that you have around 5 GB memory:
 - Matrix $(35,000 * 35,000) = 35,000 * 35,000 * 4 \approx 5 \text{ GB}$

Vector Addition

- **Example #1**

- 1 - Try to implement parallel vector addition using PyCUDA
- 2- Try to implement parallel vector addition using Numba CPU
- 3- Try to implement parallel vector addition using Numba GPU
- 4 - Compare execution times

Calculate Global Index (1D grid, 1D block)

Global Thread ID

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

threadIdx.x

threadIdx.x

threadIdx.x

threadIdx.x

0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

blockIdx.x = 0

blockIdx.x = 1

blockIdx.x = 2

blockIdx.x = 3

- Global Thread ID: $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$
- For global thread ID 26:
 - $\text{blockIdx.x} = 3$
 - $\text{blockDim.x} = 8$
 - $\text{threadIdx.x} = 2$
 - Global thread ID = $3 * 8 + 2 = 26$

Implementation via VSCODE / OOD

1. Login to OOD: <https://ondemand.snellius.surf.nl/>

2. Open the shell

3. Install packages:

```
$ module load 2024
```

```
$ module load CUDA/12.6.0
```

```
$ module load Python/3.12.3-GCCcore-13.3.0
```

```
$ pip install pycuda
```

```
$ pip install numba
```

```
$ mkdir gpucourse
```

```
$ cd gpu course
```

```
$ wget https://github.com/sara-nl/Parallel-and-GPU-programming-in-Python/archive/refs/heads/main.zip
```

```
$ unzip main
```

4. Launch a VSCODE session

PyCUDA Implementation

- Implement vector addition in PyCUDA
- Compare its execution time to the CPU version (sequential and parallel)

Numba Implementation

1. Implement vector addition in Numba
2. Compare its execution time to the CPU version (sequential and parallel)
3. Compare its performance and style to the Pycuda version

When Numba is Recommended?

- Scientific code: numerically orientated
- Code is based on numerical Python (numpy)
- Data structures are type-consistent, not mixed
- Code contains several loops that iterate over data
- Development time more important than tuning performance and efficiency

Automatic Data Transfer

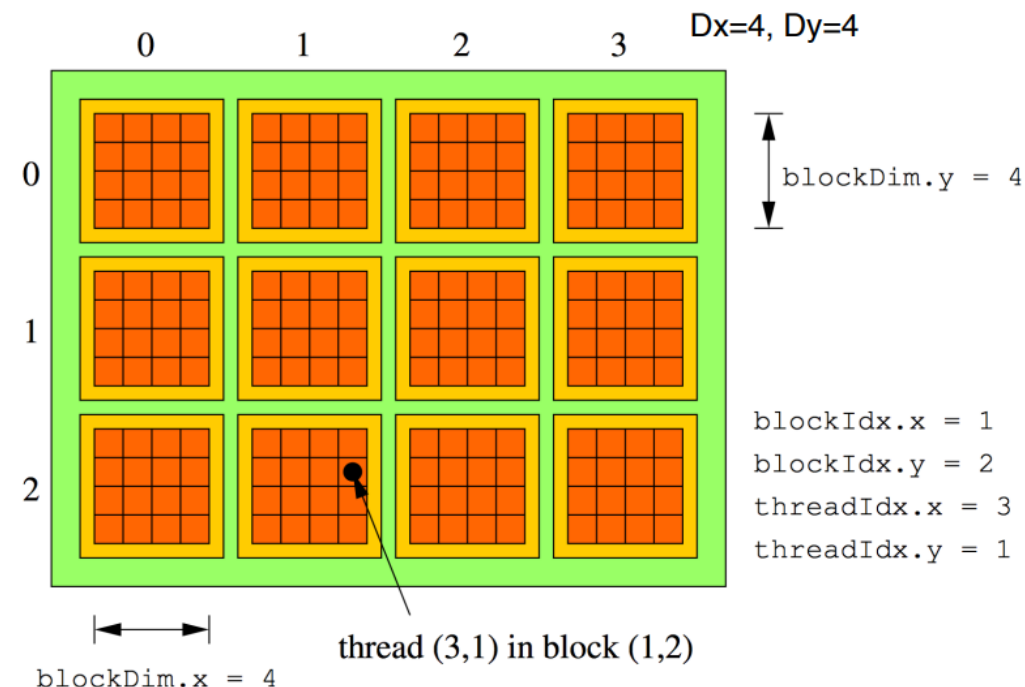
- Automatic data transfer using PyCUDA driver:
 - In()
 - Out()
 - InOut()
- PyCUDA programs become simpler

Matrix Addition

- **Example #2:**
 - 1 - Try to implement parallel Matrix (2D array) Addition using PyCUDA
 - 2- Try to implement parallel Matrix (2D array) Addition using Numba CPU
 - 3- Try to implement parallel Matrix (2D array) Addition using Numba GPU
 - 4 - Compare execution times

Calculate Global Index (2D grid, 2D block)

- Matrix 12*16



- Global Thread ID:

- $\text{row} = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y} = 2 * 4 + 1 = 9$
- $\text{column} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x} = 1 * 4 + 3 = 7$

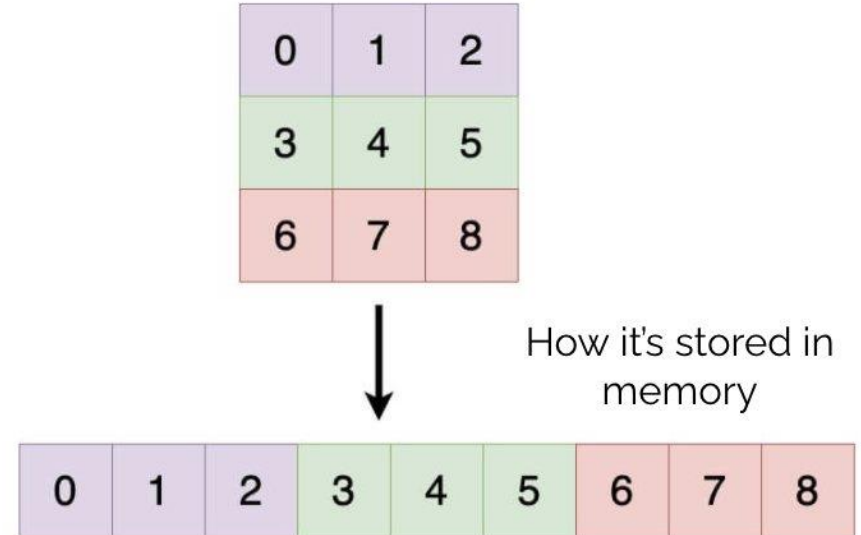
Row-Major Flattening of a Matrix

- Matrix 3*3
- For each element (row, col):
 - $\text{New ID} = \text{row} * (\text{No of col}) + \text{col}$
- For instance element “5” in location (1, 2):
 - $\text{New ID} = 1 * 3 + 2 = 5$

How we see a 2D array

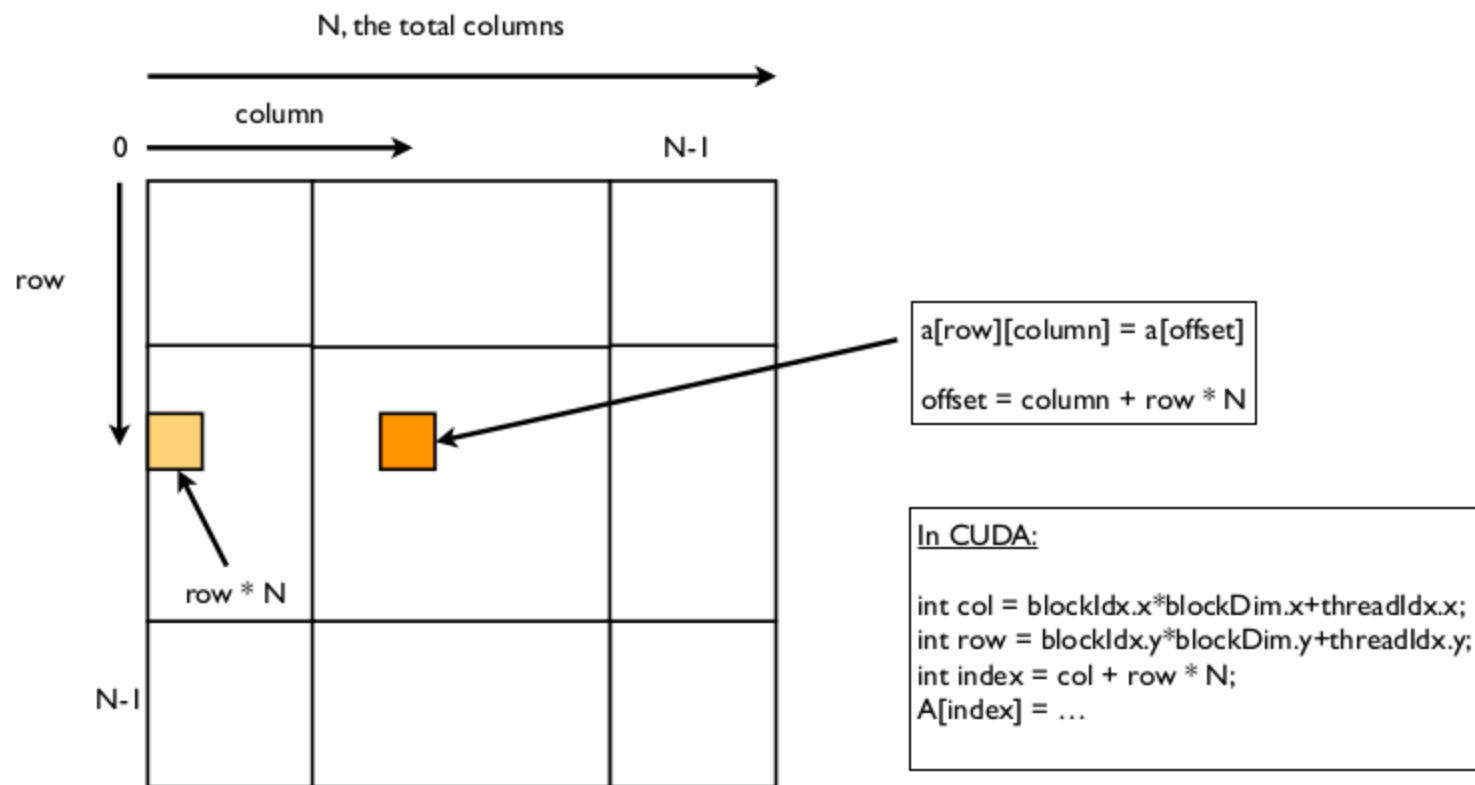
0	1	2
3	4	5
6	7	8

How it's stored in memory



0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

Row-Major Flattening of a Matrix



PyCUDA Implementation

- Implement matrix addition in PyCUDA
- Compare its execution time to the CPU version (sequential and parallel)

Numba Implementation

- Implement matrix addition in Numba
- Compare its execution time to the CPU version (sequential and parallel)
- Compare its performance and style to the Pycuda version

Matrix Transposition

- **Example #3**

- 1 - Try to transpose a matrix in parallel using PyCUDA
- 2 - Try to transpose a matrix in parallel using Numba GPU
- 3 - Try to transpose a matrix in parallel using Numba CPU
- 4 - Compare execution times

Matrix Matrix Multiplication

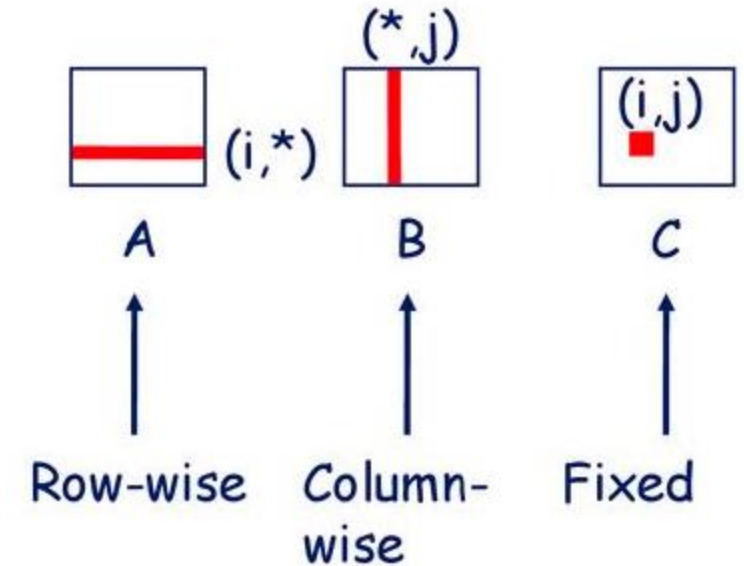
- **Example #4**

- 1 - Try to implement matrix multiplication in parallel using PyCUDA
- 2 - Try to implement matrix multiplication in parallel using Numba GPU
- 3 - Try to implement matrix multiplication in parallel using Numba CPU
- 4 - Compare execution times
 - Consider the @ operator

Sequential Matrix Multiplication

```
for i in range(N):  
    for j in range(N):  
        for k in range(N):  
            C[i][j] += A[i][k] * B[k][j]
```

Inner loop:



$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

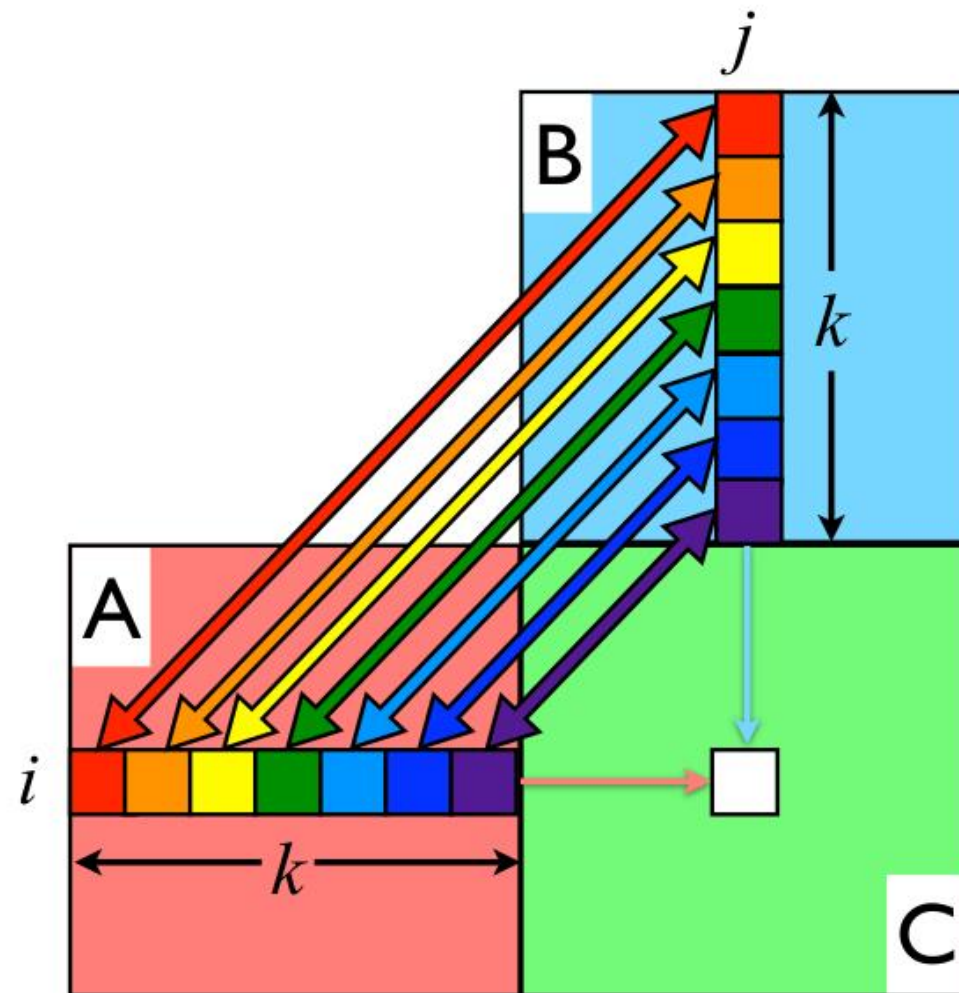
A B C

Parallel Matrix Multiplication

```
int k, sum = 0;

int col = threadIdx.x + blockDim.x * blockIdx.x;
int row = threadIdx.y + blockDim.y * blockIdx.y;
int id = row * N + col;

if (col < N && row < N){
    for(k = 0; k < N; k++){
        sum += A[row * N + k] * B[k * N + col];
    }
    C[id] = sum;
}
```

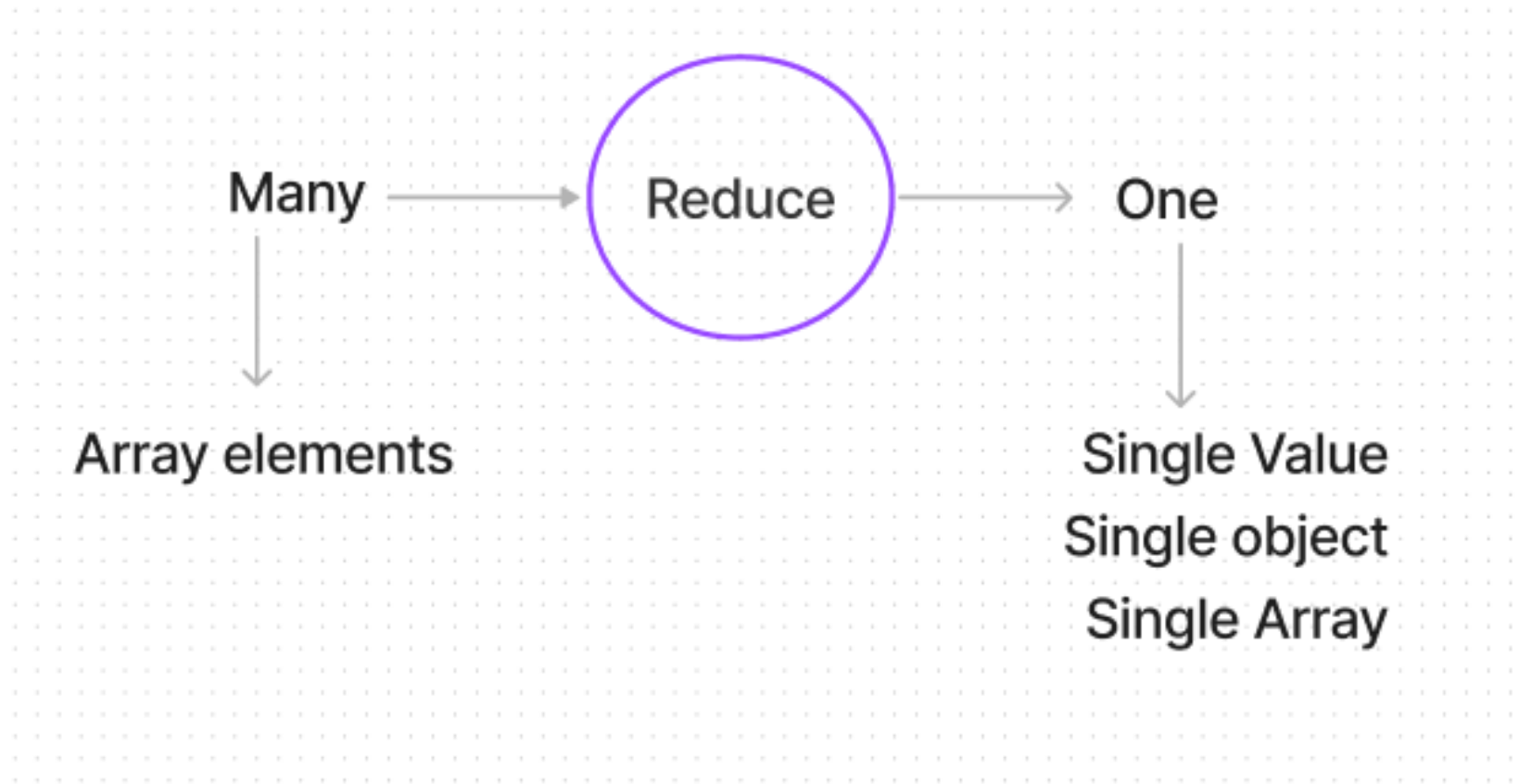


Reduction

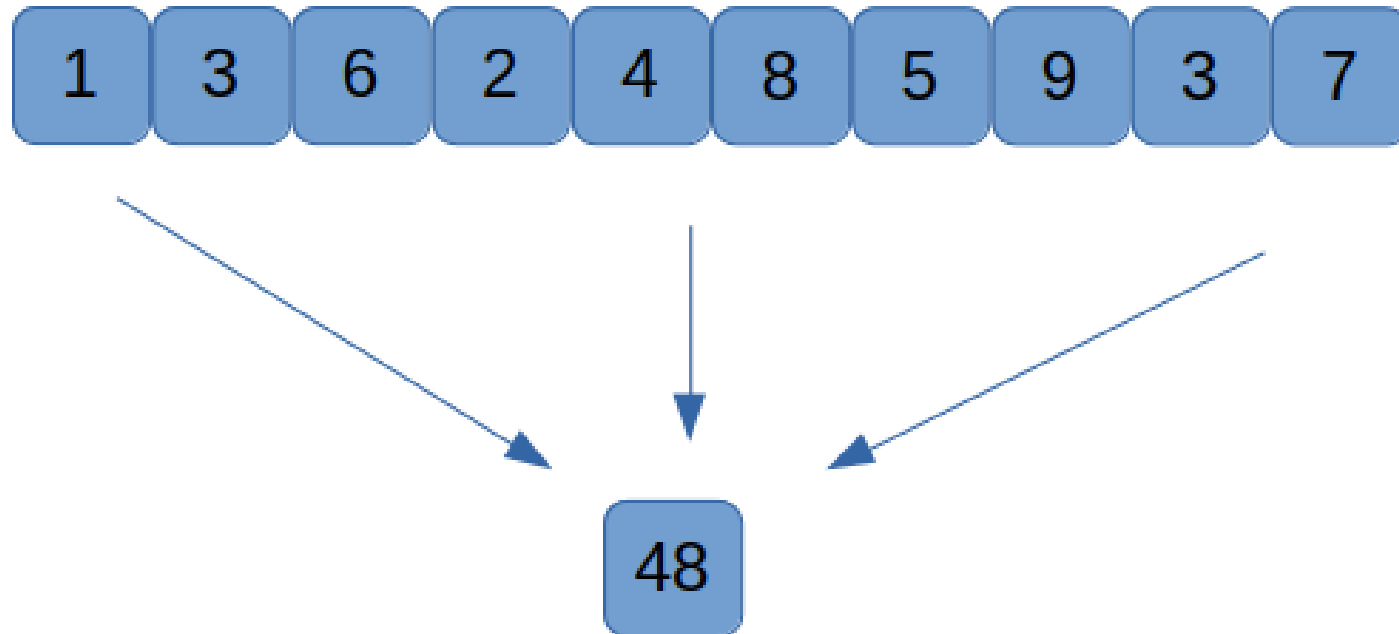
- **Example #5**

- 1 - Try to implement reduction in parallel using PyCUDA using one thread block
 - Extend it to use arbitrary size (i.e., multiple thread blocks)
- 2- Try to implement reduction in parallel using Numba CPU
- 3- Try to implement reduction in parallel using Numba GPU
- 4 - Compare execution times
 - Consider the Python reduce operator

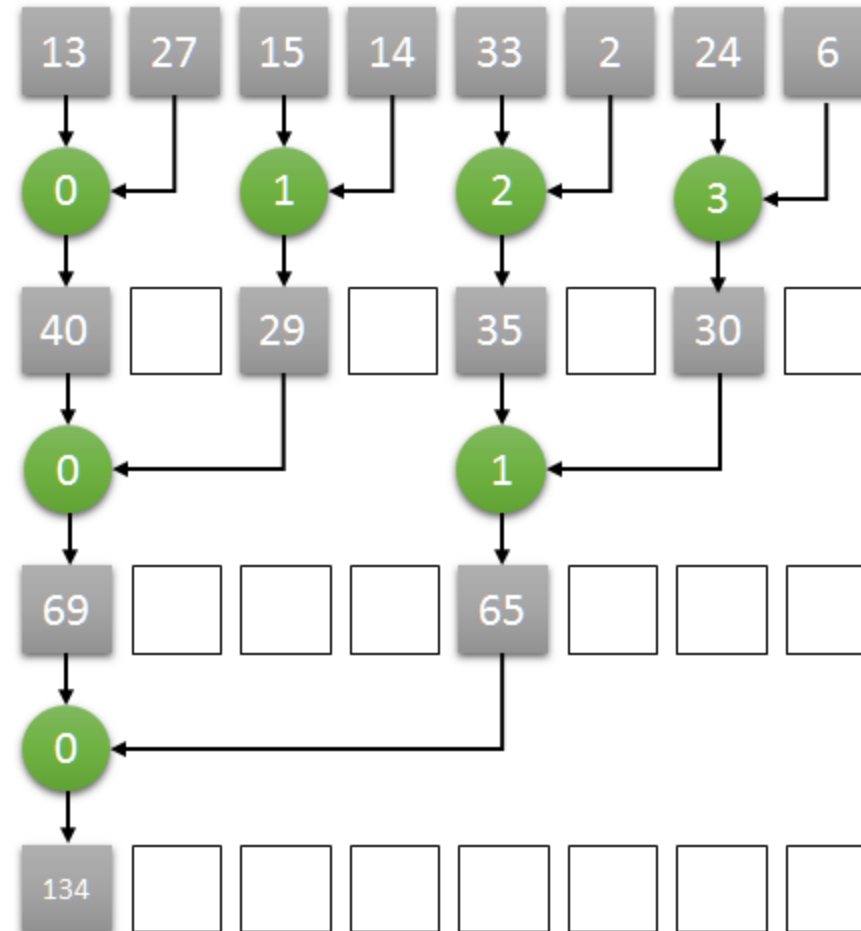
Reduction



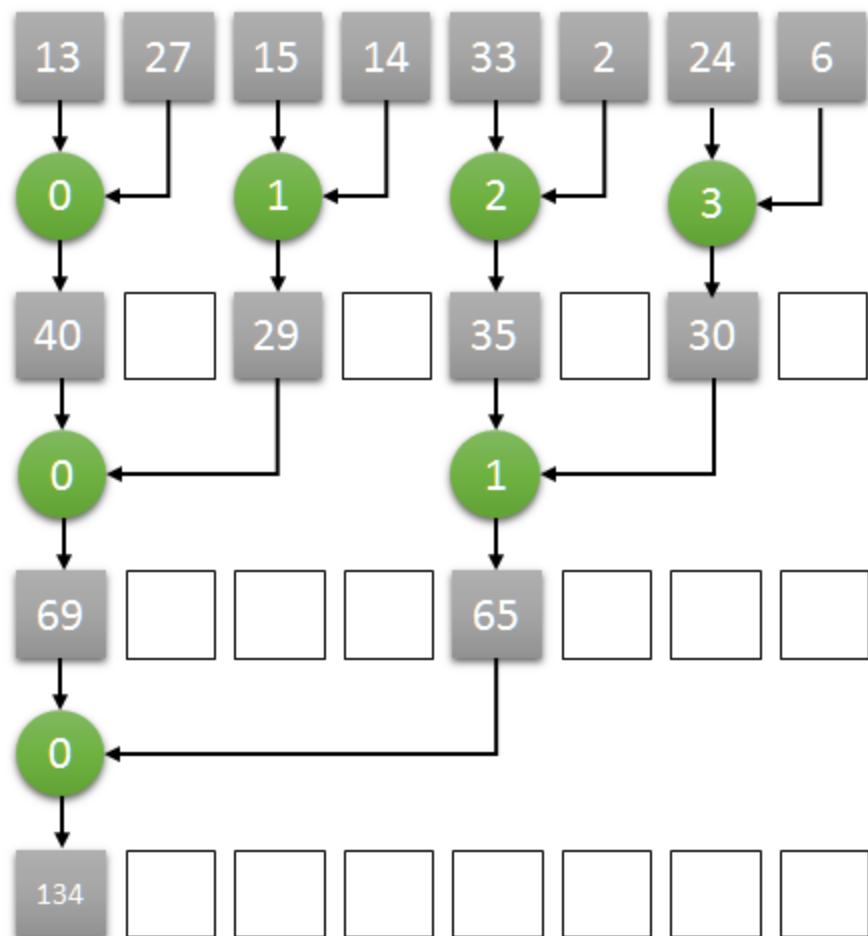
Reduction (addition)



Reduction (addition)



Reduction (addition)



level

1

$(tid \% 2 == 0) ==> a[tid] += a[tid+1]$

2

$(tid \% 4 == 0) ==> a[tid] += a[tid+2]$

3

$(tid \% 8 == 0) ==> a[tid] += a[tid+4]$

4

$(tid \% 16 == 0) ==> a[tid] += a[tid+8]$

$(tid \% (2^{\text{level}}) == 0) ==> a[tid] += a[tid+2^{(\text{level}-1)}]$

Reduction

- **Example #5**

- 1 - Try to implement reduction in parallel using PyCUDA using one thread block
 - Extend it to use arbitrary size (i.e., multiple thread blocks)
- 2- Try to implement reduction in parallel using Numba CPU
- 3- Try to implement reduction in parallel using Numba GPU
- 4 - Compare execution times
 - Consider the Python reduce operator

Reduction

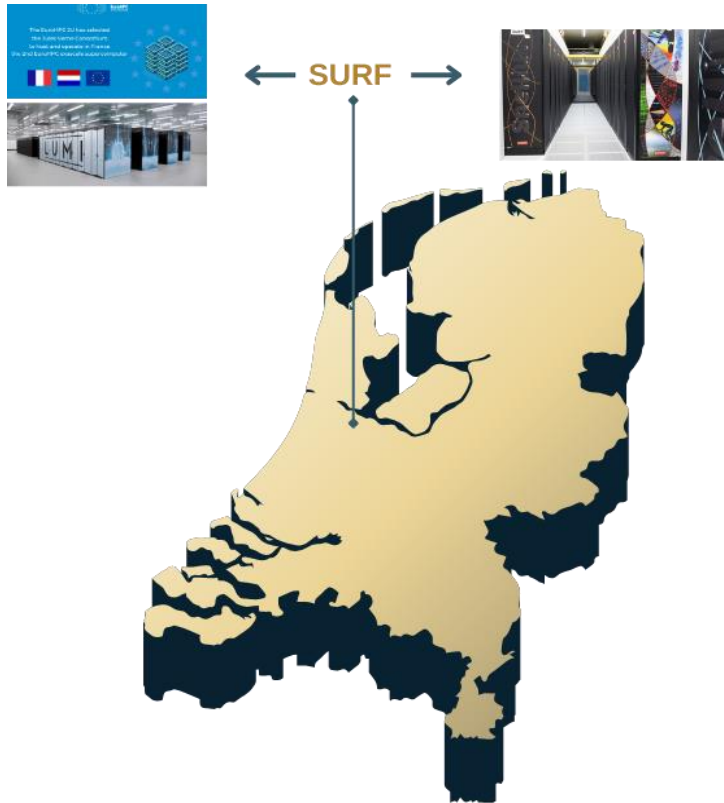
- How to use shared memory in reduction?
- Compare its execution time to the sequential version and Python reduce operator

Thank You

Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Germany, Bulgaria, Austria, Croatia, Cyprus, Czech Republic, Denmark, Estonia, Finland, Greece, Hungary, Ireland, Italy, Lithuania, Latvia, Poland, Portugal, Romania, Slovenia, Spain, Sweden, France, Netherlands, Belgium, Luxembourg, Slovakia, Norway, Türkiye, Republic of North Macedonia, Iceland, Montenegro, Serbia under grant agreement No 101101903.

Access to Supercomputers and GPUs

Current



Future

- Alice Recoque Exascale System (France - 2026)
The Alice Recoque supercomputer will be hosted and operated by the Jules Verne consortium, led by France through GENCI and CEA, with the participation of the Netherlands through SURF.
- AI Factory Groningen - 2027
consortium partners SURF, TNO and Samenwerking Noord, has taken the initiative for this facility and is working on its implementation, aiming to have it fully operational by 2027.

Supercomputers

EuroHPC Systems



✓ PROCURED

5 PETASCALE

- Vega in Slovenia
- Karolina in Czechia
- Discoverer in Bulgaria
- Meluxina in Luxembourg
- Deucalion in Portugal

3 PRE-EXASCALE

- Lumi in Finland
- Leonardo in Italy
- Marenostrum 5 in Spain

⌚ ONGOING

2 EXASCALE

- Jupiter in Germany
- Alice Recoque in France

2 MID-RANGE

- Arrhenius in Sweden
- Daedalus in Greece

▶▶ COMING NEXT

UPGRADES

- Discoverer+
- Lisa/Leonardo

AN INDUSTRIAL SYSTEM

- Co-owned and for use by the industrial sector
- For AI and other applications

A POST-EXASCALE SYSTEM

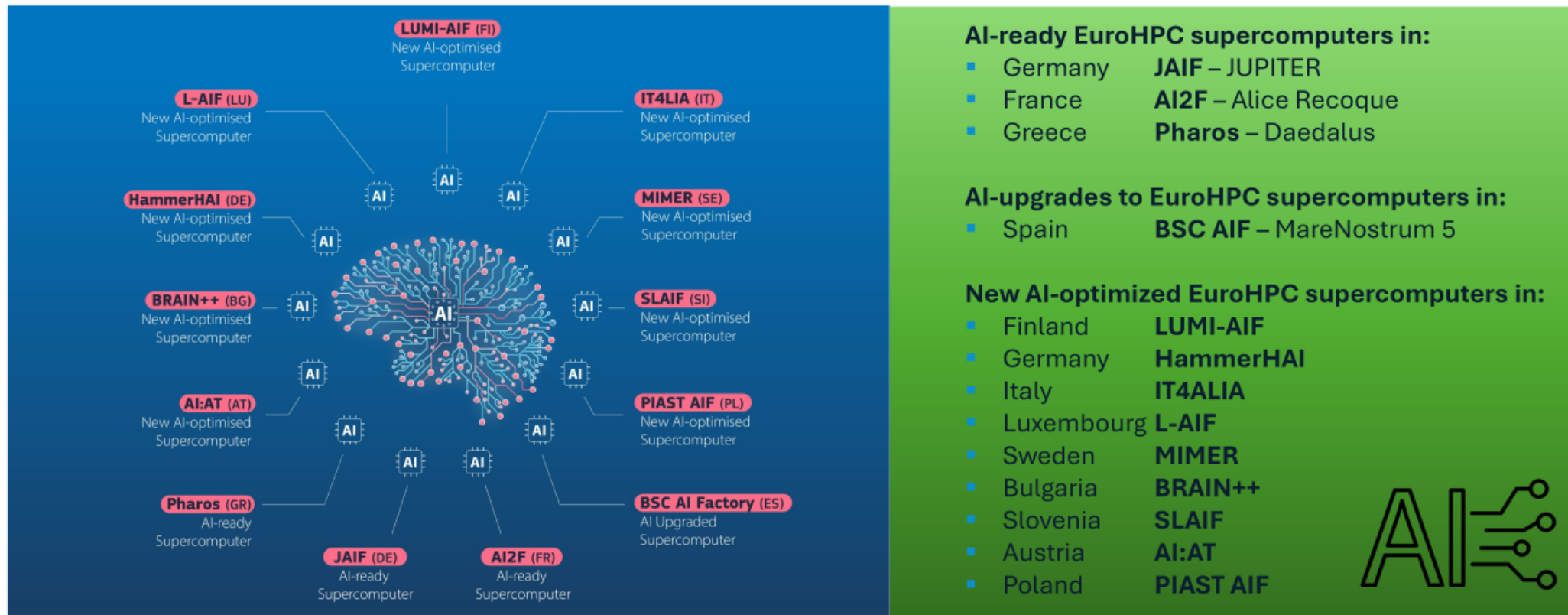
PROCUREMENT OF FEDERATION SERVICES

- A platform for the federation of EuroHPC HPC and quantum infrastructure
- A one-stop shop access point for users

Supercomputers AI factories (EuroHPC Systems)

THE AI FACTORIES

EuroHPC JU selected 13 EU sites that will host the first AI Factories – to drive Europe's leadership in AI



*AI Factories pull together EU and national resources, in a collaborative effort of **21 European countries***